

```

# === File: app/main.py ===
# AgentFlow-Eval V1.0.0
"""FastAPI application entry point with production middleware stack."""

from __future__ import annotations

from contextlib import asynccontextmanager
from typing import Any

from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware
from fastapi.middleware.trustedhost import TrustedHostMiddleware
from fastapi.responses import JSONResponse

from app.api.v1.router import router as v1_router
from app.config import settings
from app.utils.exceptions import AgentFlowError, error_response
from app.utils.logger import setup_logging
from app.core.middleware import (
    APIKeyAuthMiddleware,
    RequestIDMiddleware,
    SecurityHeadersMiddleware,
)
from app.core.observability.metrics import MetricsMiddleware, get_metrics_response
from app.core.settings_guard import enforce_production_settings

setup_logging()

try:
    from slowapi import Limiter, _rate_limit_exceeded_handler
    from slowapi.util import get_remote_address

    HAS_SLOWAPI = True
except ImportError:
    HAS_SLOWAPI = False

# ---- Lifespan events ----
def _sqlite_table_columns(connection, table: str) -> set[str]:
    from sqlalchemy import text

    rows = connection.execute(text(f"PRAGMA table_info({table})")).fetchall()
    return {row[1] for row in rows}

def _ensure_sqlite_columns(connection) -> None:
    """Add missing columns on existing SQLite DBs (create_all never ALTERs)."""
    from sqlalchemy import text
    import logging

    log = logging.getLogger(__name__)
    if connection.dialect.name != "sqlite":
        return

    task_cols = {
        "celery_task_id": "ALTER TABLE tasks ADD COLUMN celery_task_id VARCHAR(255)",
        "is_archived": "ALTER TABLE tasks ADD COLUMN is_archived BOOLEAN NOT NULL DEFAULT 0",
        "created_by": "ALTER TABLE tasks ADD COLUMN created_by VARCHAR(100) NOT NULL DEFAULT 'anonymous'",
        "tenant_id": "ALTER TABLE tasks ADD COLUMN tenant_id VARCHAR(36)",
    }

    existing = _sqlite_table_columns(connection, "tasks")
    for name, ddl in task_cols.items():
        if name not in existing:
            connection.execute(text(ddl))
            log.info("SQLite backfill: tasks.%s", name)

    if "traces" in {
        r[0]
        for r in connection.execute(
            text("SELECT name FROM sqlite_master WHERE type='table'")

```

```

    ).fetchall()
}:
    trace_cols = {
        "prompt_tokens": "ALTER TABLE traces ADD COLUMN prompt_tokens INTEGER NOT NULL DEFAULT 0",
        "completion_tokens": "ALTER TABLE traces ADD COLUMN completion_tokens INTEGER NOT NULL DEFAULT 0",
        "cost": "ALTER TABLE traces ADD COLUMN cost FLOAT NOT NULL DEFAULT 0",
        "agent_version": "ALTER TABLE traces ADD COLUMN agent_version VARCHAR(100)",
        "prompt_version": "ALTER TABLE traces ADD COLUMN prompt_version VARCHAR(100)",
        "model_version": "ALTER TABLE traces ADD COLUMN model_version VARCHAR(100)",
        "tool_version": "ALTER TABLE traces ADD COLUMN tool_version VARCHAR(100)",
    }
    existing = _sqlite_table_columns(connection, "traces")
    for name, ddl in trace_cols.items():
        if name not in existing:
            connection.execute(text(ddl))
            log.info("SQLite backfill: traces.%s", name)

if "metric_scores" in {
    r[0]
    for r in connection.execute(
        text("SELECT name FROM sqlite_master WHERE type='table'")
    ).fetchall()
}:
    ms_cols = {
        "confidence": "ALTER TABLE metric_scores ADD COLUMN confidence FLOAT",
        "is_human_reviewed": "ALTER TABLE metric_scores ADD COLUMN is_human_reviewed BOOLEAN NOT NULL DEFAULT 0",
        "human_score": "ALTER TABLE metric_scores ADD COLUMN human_score FLOAT",
        "reviewer": "ALTER TABLE metric_scores ADD COLUMN reviewer VARCHAR(100)",
    }
    existing = _sqlite_table_columns(connection, "metric_scores")
    for name, ddl in ms_cols.items():
        if name not in existing:
            connection.execute(text(ddl))
            log.info("SQLite backfill: metric_scores.%s", name)

@asynccontextmanager
async def lifespan(app: FastAPI):
    """Manage application lifecycle."""
    from app.models.base import Base
    from app.core.dependencies import engine
    import logging

    log = logging.getLogger(__name__)

    # Fail fast on insecure production configuration
    try:
        enforce_production_settings(settings)
    except Exception:
        log.exception("Production settings validation failed")
        raise

    # Import models so metadata is complete (incl. AuditLog)
    import app.models # noqa: F401

    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)
        try:
            await conn.run_sync(_ensure_sqlite_columns)
        except Exception as exc:
            log.warning("Schema backfill skipped: %s", exc)
    log.info("Tables initialized")

    # Live activity WebSocket hub (Redis pub/sub + in-process fan-out)
    ws_started = False
    try:
        from app.core.ws_hub import start_ws_hub, stop_ws_hub

        await start_ws_hub()
        ws_started = True

```

```

except Exception as exc:
    log.warning("WS hub start failed: %s", exc)

# Optional multi-layer cache warm-up (settings / dashboard / recent tasks)
if getattr(settings, "CACHE_WARMUP_ON_STARTUP", False):
    try:
        from app.core.cache.warmup import warm_cache

        summary = await warm_cache(actor="anonymous", limit=20)
        log.info("Cache warmup: %s", summary)
    except Exception as exc:
        log.warning("Cache warmup skipped: %s", exc)

# Deploy profile: bind TaskQueue / Cache / EventBus / Metering ports
try:
    from app.core.profiles import apply_profile_from_settings, profile_status

    apply_profile_from_settings()
    log.info("Infrastructure ports: %s", profile_status())
except Exception as exc:
    log.warning("Deploy profile apply failed (using lazy defaults): %s", exc)

# Plugin system: discover directories + explicit modules
plugins_bootstrapped = False
try:
    from app.core.plugins.manager import get_plugin_manager
    from app.core.plugins.market import get_plugin_market

    if getattr(settings, "PLUGINS_ENABLED", True):
        mgr = get_plugin_manager()
        strict = bool(
            getattr(settings, "PLUGIN_STRICT_ALLOWLIST", False)
            or getattr(settings, "PLUGIN_STRICT_MODE", False)
        )
        modules = list(getattr(settings, "plugin_module_list", []) or [])
        directories = list(getattr(settings, "plugin_dir_list", []) or [])
        if strict:
            # Production hardening: only explicit modules, never directory scan
            directories = []
            if not modules:
                log.warning(
                    "PLUGIN_STRICT_MODE/ALLOWLIST=true but PLUGIN_MODULES empty — no plugins loaded"
                )
        # Optional HMAC signature gate
        try:
            from app.core.plugins.signature import (
                filter_signed_modules,
                signature_check_enabled,
            )

            if signature_check_enabled() and modules:
                modules, rejected = filter_signed_modules(modules)
                if rejected:
                    log.warning("Plugin signature rejections: %s", rejected)
        except Exception as sig_exc:
            log.warning("Plugin signature check skipped: %s", sig_exc)
        summary = mgr.bootstrap(
            enabled=True,
            directories=directories,
            modules=modules,
            auto_activate=True,
            allowlist=list(getattr(settings, "plugin_allowlist", []) or [])
            or (modules if strict else None),
        )
        plugins_bootstrapped = True
    log.info(
        "Plugins bootstrap (strict=%s): %s",
        strict,
        summary,
    )

```

```

        market = get_plugin_market()
        if getattr(settings, "PLUGIN_MARKET_SEED_EXAMPLES", True):
            market.seed_built_in_catalog()
        catalog = getattr(settings, "PLUGIN_CATALOG_PATH", "") or ""
        if catalog:
            market.catalog_path = __import__("pathlib").Path(catalog)
            market.load_catalog()
        else:
            log.info("Plugin system disabled (PLUGINS_ENABLED=false)")
    except Exception as exc:
        log.warning("Plugin bootstrap skipped: %s", exc)

yield

if plugins_bootstrapped:
    try:
        from app.core.plugins.manager import get_plugin_manager

        get_plugin_manager().shutdown()
    except Exception as exc:
        log.warning("Plugin shutdown failed: %s", exc)

if ws_started:
    try:
        from app.core.ws_hub import stop_ws_hub

        await stop_ws_hub()
    except Exception as exc:
        log.warning("WS hub stop failed: %s", exc)

await engine.dispose()
from app.core.dependencies import close_redis

await close_redis()

app = FastAPI(
    title=settings.APP_NAME,
    description="Agent automated evaluation platform",
    version="1.0.0",
    lifespan=lifespan,
    docs_url="/docs" if not settings.is_prod else None,
    redoc_url="/redoc" if not settings.is_prod else None,
)

# ---- Rate limiting ----
if HAS_SLOWAPI and settings.RATE_LIMIT_ENABLED:
    limiter = Limiter(
        key_func=get_remote_address, default_limits=[settings.RATE_LIMIT_DEFAULT]
    )
    app.state.limiter = limiter
    app.add_exception_handler(429, _rate_limit_exceeded_handler)

# Middleware order: last added = outermost. Desired stack:
# CORS -> RequestID -> SecurityHeaders -> Metrics -> APIKeyAuth -> app
app.add_middleware(APIKeyAuthMiddleware)
app.add_middleware(MetricsMiddleware)
app.add_middleware(SecurityHeadersMiddleware)
app.add_middleware(RequestIDMiddleware)
app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.CORS_ORIGINS,
    allow_credentials=True,
    allow_methods=["GET", "POST", "PUT", "DELETE", "OPTIONS"],
    allow_headers=[
        "Authorization",
        "Content-Type",
        "X-Request-ID",
        "X-Trace-ID",
        "X-API-Key",
    ],

```

```

    ],
    expose_headers=["X-Request-ID", "X-Trace-ID", "X-Error-ID"],
)

# ---- Trusted hosts ----
if settings.is_prod:
    app.add_middleware(TrustedHostMiddleware, allowed_hosts=["*"])

# ---- Routes ----
app.include_router(v1_router)

# ---- Exception handlers ----
@app.exception_handler(AgentFlowError)
async def agentflow_error_handler(
    request: Request, exc: AgentFlowError
) -> JSONResponse:
    rid = getattr(request.state, "request_id", None)
    error_id = getattr(request.state, "error_id", None)
    if not error_id and exc.status_code >= 500:
        try:
            from app.core.observability.aols import new_error_id

            error_id = new_error_id()
            request.state.error_id = error_id
        except Exception:
            error_id = None
    if exc.status_code >= 500:
        try:
            from app.core.observability.aols import LogEvent, get_logger

            get_logger("app.errors").error(
                str(LogEvent.SYSTEM_ERROR),
                error_type=type(exc).__name__,
                error_message=exc.message,
                status_code=exc.status_code,
                path=request.url.path,
                request_id=rid,
                error_id=error_id,
            )
        except Exception:
            pass
    body = error_response(
        exc.status_code,
        exc.message,
        exc.detail,
        request_id=rid,
        error_id=error_id if exc.status_code >= 500 else None,
    )
    resp = JSONResponse(status_code=exc.status_code, content=body)
    if error_id:
        resp.headers["X-Error-ID"] = str(error_id)
    return resp

@app.exception_handler(Exception)
async def global_exception_handler(request: Request, exc: Exception) -> JSONResponse:
    import traceback

    rid = getattr(request.state, "request_id", None)
    try:
        from app.core.observability.aols import LogEvent, get_logger, new_error_id

        error_id = getattr(request.state, "error_id", None) or new_error_id()
        request.state.error_id = error_id
        get_logger("app.errors").error(
            str(LogEvent.SYSTEM_ERROR),
            error_type=type(exc).__name__,
            error_message=str(exc),
            status_code=500,

```

```

        path=request.url.path,
        request_id=request_id,
        error_id=error_id,
        exc_info=True,
    )
except Exception:
    error_id = getattr(request.state, "error_id", None)

stack = traceback.format_exc() if settings.DEBUG else None
resp = JSONResponse(
    status_code=500,
    content=error_response(
        500,
        "Internal error",
        str(exc) if settings.DEBUG else None,
        request_id=request_id,
        stacktrace=stack,
        error_id=error_id,
    ),
)
if error_id:
    resp.headers["X-Error-ID"] = str(error_id)
return resp

# ---- Prometheus metrics ----
@app.get("/metrics", include_in_schema=False)
async def prometheus_metrics() -> Any:
    """Prometheus text exposition endpoint (scraped by Prometheus / Grafana Agent).

    Public path (no API key). Disable via ``METRICS_ENABLED=false`` to return 404.
    """
    if not settings.METRICS_ENABLED:
        return JSONResponse(status_code=404, content={"detail": "Metrics disabled"})
    return get_metrics_response()

# ---- Health checks (liveness / readiness / composite) ----
APP_VERSION = "1.0.0"

async def _probe_services() -> dict[str, str]:
    """Probe database and Redis; return per-service status strings."""
    from sqlalchemy import text
    from app.core.dependencies import engine

    services: dict[str, str] = {}
    try:
        async with engine.connect() as conn:
            await conn.execute(text("SELECT 1"))
            services["database"] = "ok"
    except Exception as e:
        services["database"] = f"error: {e}"

    # Lite / eager / memory queue: never block readiness on Redis
    skip_redis = bool(settings.CELERY_TASK_ALWAYS_EAGER)
    try:
        from app.core.profiles import current_profile, get_task_queue

        if current_profile() == "lite" or get_task_queue().is_eager():
            skip_redis = True
        if get_task_queue().backend_name in {"eager", "memory"}:
            skip_redis = True
    except Exception:
        pass

    if skip_redis:
        services["redis"] = "skipped (lite/eager)"
    else:
        try:

```

```

        from app.core.dependencies import get_redis

        r = await get_redis()
        await r.ping()
        services["redis"] = "ok"
    except Exception:
        services["redis"] = "unavailable"
    return services

@app.get("/health/live")
async def liveness() -> dict[str, Any]:
    """Kubernetes-style liveness probe — process is up, no dependency checks.

    Always returns HTTP 200 if the event loop can serve the request.
    """
    return {
        "status": "alive",
        "app": settings.APP_NAME,
        "version": APP_VERSION,
    }

@app.get("/health/ready")
async def readiness(request: Request) -> JSONResponse:
    """Kubernetes-style readiness probe — ready to accept traffic.

    Returns 200 when the database is reachable; 503 when critical deps fail.
    Redis is required only when not in Celery eager mode.
    """
    services = await _probe_services()
    critical_ok = services.get("database") == "ok"
    redis_ok = services.get("redis", "").startswith(("ok", "skipped"))
    ready = critical_ok and redis_ok
    deploy = {}
    try:
        from app.core.profiles import profile_status

        deploy = profile_status()
    except Exception:
        pass
    body: dict[str, Any] = {
        "status": "ready" if ready else "not_ready",
        "app": settings.APP_NAME,
        "version": APP_VERSION,
        "services": services,
        "celery_eager": settings.CELERY_TASK_ALWAYS_EAGER,
        "deploy": deploy,
    }
    return JSONResponse(status_code=200 if ready else 503, content=body)

@app.get("/health")
async def health_check() -> dict[str, Any]:
    """Composite health check with database and Redis connectivity.

    Backward-compatible with existing Docker healthchecks. Prefer
    ``/health/live`` and ``/health/ready`` for orchestrators.
    """
    services = await _probe_services()
    critical_ok = services.get("database") == "ok"
    redis_ok = services.get("redis", "").startswith(("ok", "skipped"))
    all_healthy = critical_ok and redis_ok
    deploy = {}
    try:
        from app.core.profiles import profile_status

        deploy = profile_status()
    except Exception:
        pass

```

```

    return {
        "status": "healthy" if all_healthy else "degraded",
        "app": settings.APP_NAME,
        "version": APP_VERSION,
        "services": services,
        "celery_eager": settings.CELERY_TASK_ALWAYS_EAGER,
        "deploy": deploy,
    }
# === File: app/config.py ===
# AgentFlow-Eval V1.0.0
"""Application configuration using pydantic-settings with environment modes."""

from __future__ import annotations

from pathlib import Path
from typing import Literal

from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    """Production-grade configuration with dev/test/prod separation.

    Environment variables override .env values which override defaults.
    """

    # ---- Environment ----
    ENV: Literal["dev", "test", "prod"] = "dev"
    DEBUG: bool = True
    APP_NAME: str = "AgentFlow Intelligence"
    API_V1_PREFIX: str = "/api/v1"
    SECRET_KEY: str = "change-me-in-production"

    # ---- CORS ----
    CORS_ORIGINS: list[str] = [
        "http://localhost:5173",
        "http://127.0.0.1:5173",
        "http://localhost:3000",
    ]

    # ---- Database ----
    DATABASE_URL: str = "sqlite+aiosqlite:///./agentflow_eval.db"
    DB_POOL_SIZE: int = 20
    DB_MAX_OVERFLOW: int = 40
    DB_ECHO: bool = False

    # ---- Redis / Celery ----
    REDIS_URL: str = "redis://localhost:6379/0"
    CELERY_BROKER_URL: str = "redis://localhost:6379/0"
    CELERY_RESULT_BACKEND: str = "redis://localhost:6379/0"
    CELERY_TASK_ALWAYS_EAGER: bool = False

    # ---- OpenAI ----
    OPENAI_API_KEY: str = ""
    OPENAI_BASE_URL: str = "https://api.openai.com/v1"

    # ---- Logging (AOLS) ----
    LOG_LEVEL: str = "INFO"
    LOG_FILE: str = "logs/agentflow_eval.log"
    # Prefer "json" in prod; text keeps human-readable local dev consoles
    LOG_FORMAT: Literal["json", "text"] = "text"
    LOG_MAX_BYTES: int = 10485760 # 10 MB
    LOG_BACKUP_COUNT: int = 30
    LOG_SERVICE_NAME: str = "agentflow-api"
    # Skip high-frequency paths in access logs (comma-separated prefixes)
    LOG_ACCESS_SKIP_PATHS: str = "/health,/metrics,/favicon.ico"
    # Max string length when redacting nested log values
    LOG_REDACT_MAX_STR: int = 2000
    # Persist structured events to agent_logs table (best-effort batch)
    LOG_DB_SINK: bool = True

```



```

LOG_DB_BATCH_SIZE: int = 32
LOG_DB_FLUSH_INTERVAL_SEC: float = 1.0
# Retention hint for cleanup jobs (days); 0 = no auto cleanup in v1
LOG_DB_RETENTION_DAYS: int = 30

# ---- Rate Limiting ----
RATE_LIMIT_ENABLED: bool = True
RATE_LIMIT_DEFAULT: str = "100/minute"

# ---- Auth (API Key) ----
# When AUTH_ENABLED=true, all /api/v1/* routes require X-API-Key or Bearer token.
# API_KEYS examples:
#   "dev-secret"
#   "dev-secret:alice"
#   "dev-secret:alice:manager,prod-secret:bob:user"
AUTH_ENABLED: bool = False
API_KEYS: str = ""

# ---- RBAC ----
# When true (and AUTH_ENABLED), enforce role permissions on API endpoints.
RBAC_ENABLED: bool = True
# Default role for actors without ACTOR_ROLES / key-embedded role.
# Enterprise default: member (legacy "user" still accepted via Role.parse).
DEFAULT_ROLE: str = "member"
# Optional actor→role map: "alice:manager,bob:reviewer"
ACTOR_ROLES: str = ""

# ---- Tenancy (actor isolation) ----
# When AUTH_ENABLED or TENANCY_ENABLED is true, tasks are scoped by created_by.
TENANCY_ENABLED: bool = False
# Comma-separated actors that can see all tasks, e.g. "admin,ops"
# These actors also resolve to Role.SYSTEM_ADMIN when no explicit role is set.
ADMIN_ACTORS: str = "admin"

# ---- Enterprise multi-tenant (org tables + X-Tenant-ID) ----
# When true: resolve X-Tenant-ID, enforce membership, filter by tenant_id.
# Lite/private demos keep this false (actor isolation only).
MULTI_TENANT_ENABLED: bool = False

# ---- Tool sandbox ----
TOOL_TIMEOUT_SEC: float = 3.0

# ---- HTTP Agent (Phase 1) ----
# When False (default), block private/loopback/link-local targets (SSRF guard).
# Set True only for controlled lab tests against local mock agents.
HTTP_AGENT_ALLOW_PRIVATE_IP: bool = False
HTTP_AGENT_DEFAULT_TIMEOUT_SEC: float = 60.0
# When True, resolve hostnames and reject private A/AAAA (DNS rebinding defense)
HTTP_AGENT_SSRF_RESOLVE_DNS: bool = True

# ---- Request Limits ----
MAX_REQUEST_SIZE: int = 10 * 1024 * 1024 # 10 MB

# ---- Pagination ----
DEFAULT_PAGE_SIZE: int = 20
MAX_PAGE_SIZE: int = 100

# ---- Unicorn ----
GUNICORN_WORKERS: int = 4
GUNICORN_WORKER_CLASS: str = "uvicorn.workers.UvicornWorker"
GUNICORN_TIMEOUT: int = 120

# ---- Judge engine ----
# Soft timeout (seconds) for a single evaluate() call; 0 disables.
JUDGE_TIMEOUT_SEC: float = 60.0
# In-process result cache size (LRU).
JUDGE_CACHE_SIZE: int = 128

# ---- Observability (Prometheus) ----
METRICS_ENABLED: bool = True

```

```

# ---- Cache (L1 memory + L2 Redis) ----
CACHE_ENABLED: bool = True
CACHE_WARMUP_ON_STARTUP: bool = False

# ---- Resilience (LLM / external calls) ----
LLM_RETRY_ENABLED: bool = True
LLM_MAX_RETRIES: int = 3
LLM_RETRY_MIN_WAIT_SEC: float = 1.0
LLM_RETRY_MAX_WAIT_SEC: float = 10.0
LLM_CALL_TIMEOUT_SEC: float = 30.0
CIRCUIT_ENABLED: bool = True
CIRCUIT_FAILURE_THRESHOLD: int = 5
CIRCUIT_RECOVERY_TIMEOUT_SEC: float = 60.0

# ---- Multimodal storage & vision ----
# local | s3 | minio
STORAGE_BACKEND: str = "local"
LOCAL_STORAGE_PATH: str = "data/uploads"
S3_BUCKET: str = "agentflow"
S3_ENDPOINT_URL: str = "" # e.g. http://localhost:9000 for MinIO
S3_ACCESS_KEY: str = ""
S3_SECRET_KEY: str = ""
S3_REGION: str = "us-east-1"
S3_PREFIX: str = ""
# Max upload size for multimodal files (bytes)
MEDIA_MAX_UPLOAD_BYTES: int = 20 * 1024 * 1024 # 20 MB
# Vision multimodal model (OpenAI-compatible)
VISION_MODEL: str = "gpt-4o-mini"

# ---- Deploy profiles (lite | private | saas | auto) ----
# lite: SQLite + eager queue + memory cache + in-process bus (zero Redis)
# private: full middleware (default)
# saas: private + billing hooks enabled when BILLING_ENABLED
# auto: lite if sqlite+eager else private
DEPLOY_PROFILE: str = "auto"
# celery | eager | memory (empty = profile default)
TASK_QUEUE_BACKEND: str = ""
# SaaS metering (tables land in later migration; noop until then)
BILLING_ENABLED: bool = False
# Stripe Checkout (mock by default — no real charges without keys)
# mock | live
STRIPE_MODE: str = "mock"
STRIPE_SECRET_KEY: str = ""
STRIPE_WEBHOOK_SECRET: str = ""
STRIPE_SUCCESS_URL: str = "http://localhost:5173/billing?checkout=success"
STRIPE_CANCEL_URL: str = "http://localhost:5173/billing?checkout=cancel"
# Optional price id map: "pro:price_xxx,enterprise:price_yyy"
STRIPE_PRICE_IDS: str = ""
# Slow-task threshold for observability sampling (seconds)
SLOW_TASK_THRESHOLD_SEC: float = 30.0
OBSERVABILITY_KPI_ENABLED: bool = True

# ---- Plugins ----
# Master switch for discovery/load on startup.
PLUGINS_ENABLED: bool = True
# Comma-separated directories to scan (relative to CWD or absolute).
# Example: "plugins,./external_plugins"
PLUGIN_DIRS: str = "plugins"
# Comma-separated Python entries: "pkg.mod:Plugin,app.plugins.examples.echo_tool:Plugin"
PLUGIN_MODULES: str = ""
# Optional local marketplace catalog JSON path.
PLUGIN_CATALOG_PATH: str = ""
# When true, seed example plugins into the in-memory market catalog.
PLUGIN_MARKET_SEED_EXAMPLES: bool = True
# When true, only load modules listed in PLUGIN_MODULES (ignore PLUGIN_DIRS scan).
# Production hardening: set PLUGIN_MODULES=pkg:Plugin,... and PLUGIN_STRICT_ALLOWLIST=true
PLUGIN_STRICT_ALLOWLIST: bool = False
# Alias of PLUGIN_STRICT_ALLOWLIST (enterprise naming)
PLUGIN_STRICT_MODE: bool = False

```

```

# Optional extra allowlist of module prefixes or full entries (comma-separated).
# Empty = only PLUGIN_MODULES when strict; when not strict, no extra filter.
PLUGIN_ALLOWLIST: str = ""
# When true, require HMAC signatures for PLUGIN_MODULES (see PLUGIN_SIGNATURES)
PLUGIN_SIGNATURE_CHECK: bool = False
# HMAC key for plugin file integrity
PLUGIN_SIGNING_SECRET: str = ""
# "module.path:hexdigest,other:hexdigest"
PLUGIN_SIGNATURES: str = ""

@property
def plugin_dir_list(self) -> list[str]:
    raw = (self.PLUGIN_DIRS or "").strip()
    if not raw:
        return []
    return [p.strip() for p in raw.replace(";", " ").split(",") if p.strip()]

@property
def plugin_module_list(self) -> list[str]:
    raw = (self.PLUGIN_MODULES or "").strip()
    if not raw:
        return []
    return [p.strip() for p in raw.replace(";", " ").split(",") if p.strip()]

@property
def plugin_allowlist(self) -> list[str]:
    raw = (self.PLUGIN_ALLOWLIST or "").strip()
    if not raw:
        return []
    return [p.strip() for p in raw.replace(";", " ").split(",") if p.strip()]

@property
def is_dev(self) -> bool:
    return self.ENV == "dev"

@property
def is_test(self) -> bool:
    return self.ENV == "test"

@property
def is_prod(self) -> bool:
    return self.ENV == "prod"

model_config = SettingsConfigDict(
    env_file=".env",
    env_file_encoding="utf-8",
    extra="ignore",
    case_sensitive=True,
)

settings = Settings()

# Ensure log directory exists
Path(settings.LOG_FILE).parent.mkdir(parents=True, exist_ok=True)
# === File: app/core/dependencies.py ===
# AgentFlow-Eval V1.0.0
"""Dependency injection with database pool and Redis cache."""

from __future__ import annotations

from collections.abc import AsyncGenerator
from typing import Any

import redis.asyncio as aioredis
from sqlalchemy.ext.asyncio import (
    AsyncSession,
    async_sessionmaker,
    create_async_engine,
)

```

```

from app.config import settings

# ---- Database engine with connection pool ----
_engine_kw: dict = {
    "echo": settings.DB_ECHO,
    "future": True,
}
_db_url = settings.DATABASE_URL or ""
if "postgresql" in _db_url:
    _engine_kw.update(
        {
            "pool_size": settings.DB_POOL_SIZE,
            "max_overflow": settings.DB_MAX_OVERFLOW,
            "pool_pre_ping": True,
            "pool_recycle": 3600,
        }
    )
elif "sqlite" in _db_url:
    # SQLite: avoid cross-thread issues; enable WAL via connect event below
    _engine_kw.update(
        {
            "connect_args": {"check_same_thread": False},
        }
    )

engine = create_async_engine(settings.DATABASE_URL, **_engine_kw)

# SQLite performance pragmas (WAL + reasonable sync for local/dev)
if "sqlite" in _db_url:
    from sqlalchemy import event

    @event.listens_for(engine.sync_engine, "connect")
    def _sqlite_on_connect(dbapi_conn, connection_record) -> None: # type: ignore[no-untyped-def]
        cursor = dbapi_conn.cursor()
        try:
            cursor.execute("PRAGMA journal_mode=WAL")
            cursor.execute("PRAGMA synchronous=NORMAL")
            cursor.execute("PRAGMA foreign_keys=ON")
            cursor.execute("PRAGMA temp_store=MEMORY")
            cursor.execute("PRAGMA busy_timeout=5000")
        finally:
            cursor.close()

async_session_factory = async_sessionmaker(
    engine,
    class_=AsyncSession,
    expire_on_commit=False,
)

async def get_db() -> AsyncGenerator[AsyncSession, Any]:
    """Provide an async database session with auto-commit/rollback."""
    async with async_session_factory() as session:
        try:
            yield session
            await session.commit()
        except Exception:
            await session.rollback()
            raise
        finally:
            await session.close()

async def bind_tenant(request: Any, session: AsyncSession) -> Any:
    """Resolve X-Tenant-ID against DB membership; store on request.state.

    Call from endpoints after injecting ``Request`` + ``get_db`` session, or

```

```

wrap with ``Depends`` via a thin FastAPI-layer helper in routers.
"""

from app.core.tenant_context import extract_tenant_header, resolve_tenant_context

actor = getattr(request.state, "actor", None) or "anonymous"
role = getattr(request.state, "role", None)
role_s = role.value if hasattr(role, "value") else (str(role) if role else "")
header = extract_tenant_header(request)
ctx = await resolve_tenant_context(
    session,
    actor=actor,
    header_value=header,
    system_role=role_s,
)
request.state.tenant_id = ctx.tenant_id
request.state.tenant_slug = ctx.tenant_slug
request.state.tenant_member_role = ctx.member_role
return ctx

# ---- Redis cache ----
_redis: aioredis.Redis | None = None

async def get_redis() -> aioredis.Redis:
    """Lazy-init Redis connection pool."""
    global _redis
    if _redis is None:
        _redis = aioredis.from_url(
            settings.REDIS_URL,
            encoding="utf-8",
            decode_responses=True,
            # Fail fast when Redis is down (local eager mode / offline)
            socket_timeout=0.8,
            socket_connect_timeout=0.5,
        )
    return _redis

async def close_redis() -> None:
    """Close the Redis connection pool gracefully."""
    global _redis
    if _redis:
        await _redis.close()
        _redis = None

# ---- Cache helpers (compat wrappers → app.core.cache) ----
CACHE_TTL_DEFAULT = 300 # 5 minutes
CACHE_TTL_LIST = 60 # 1 minute for lists

async def cache_get(key: str) -> Any | None:
    """Get value from multi-layer cache (L1 + Redis)."""
    from app.core.cache.client import get_cache

    return await get_cache().get(key)

async def cache_set(key: str, value: Any, ttl: int = CACHE_TTL_DEFAULT) -> None:
    """Set value in multi-layer cache with TTL."""
    from app.core.cache.client import get_cache

    await get_cache().set(key, value, ttl=ttl)

async def cache_invalidate(pattern: str) -> None:
    """Invalidate all keys matching a pattern."""
    from app.core.cache.client import get_cache

```

```

        await get_cache().delete_pattern(pattern)
# === File: app/models/__init__.py ===
# AgentFlow-Eval V1.0.0
"""数据模型包, 包含 Task、TestSuite、Trace、MetricScore 等 ORM 模型。"""

from app.models.base import Base
from app.models.task import Task
from app.models.test_suite import TestSuite
from app.models.trace import Trace
from app.models.metric_score import MetricScore
from app.models.audit_log import AuditLog
from app.models.experiment import Experiment, ExperimentRun
from app.models.media_asset import MediaAsset
from app.models.ab_test import ABAssignment, ABEvent, ABExperiment, ABVariant
from app.models.billing import (
    BillingPlan,
    Invoice,
    QuotaBalance,
    Subscription,
    UsageRecord,
)
from app.models.slow_task import SlowTaskEvent
from app.models.agent_log import AgentLog
from app.models.tenant import Tenant, TenantMember
from app.models.benchmark import (
    Benchmark,
    BenchmarkCase,
    BenchmarkResult,
    BenchmarkRun,
)

__all__ = [
    "Base",
    "Task",
    "TestSuite",
    "Trace",
    "MetricScore",
    "AuditLog",
    "Experiment",
    "ExperimentRun",
    "MediaAsset",
    "ABExperiment",
    "ABVariant",
    "ABAssignment",
    "ABEvent",
    "BillingPlan",
    "Subscription",
    "UsageRecord",
    "QuotaBalance",
    "Invoice",
    "SlowTaskEvent",
    "AgentLog",
    "Tenant",
    "TenantMember",
    "Benchmark",
    "BenchmarkCase",
    "BenchmarkRun",
    "BenchmarkResult",
]
# === File: app/models/ab_test.py ===
# AgentFlow-Eval V1.0.0
"""A/B test models — online experiments with traffic split & events."""

from __future__ import annotations

import enum
from datetime import datetime
from typing import TYPE_CHECKING, Any

from sqlalchemy import (

```

```

Boolean,
DateTime,
Float,
ForeignKey,
Index,
Integer,
JSON,
String,
Text,
UniqueConstraint,
)
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.models.base import Base, PKMixin, TenantMixin, TimestampMixin

if TYPE_CHECKING:
    pass

class ABStatus(str, enum.Enum):
    """Lifecycle of an online A/B experiment."""

    DRAFT = "draft"
    RUNNING = "running"
    PAUSED = "paused"
    COMPLETED = "completed"
    ARCHIVED = "archived"

class ABExperiment(PKMixin, TenantMixin, TimestampMixin, Base):
    """Online A/B experiment with sticky assignment and event logging.

    Complements offline ``Experiment`` (batch suite comparison):
    - AB: continuous traffic split, exposures/conversions, significance
    - Experiment: one-shot multi-config eval on fixed suites
    """

    __tablename__ = "ab_experiments"
    __table_args__ = (
        UniqueConstraint("key", name="uq_ab_experiments_key"),
        Index("ix_ab_experiments_status", "status"),
        Index("ix_ab_experiments_created_by", "created_by"),
        Index("ix_ab_experiments_tenant", "tenant_id"),
    )

    key: Mapped[str] = mapped_column(
        String(100),
        nullable=False,
        comment="Stable public key used in assign API, e.g. judge_prompt_v2",
    )
    name: Mapped[str] = mapped_column(String(255), nullable=False)
    description: Mapped[str] = mapped_column(Text, nullable=False, default="")
    status: Mapped[str] = mapped_column(
        String(20), nullable=False, default=ABStatus.DRAFT.value
    )

    # Statistical config
    alpha: Mapped[float] = mapped_column(Float, nullable=False, default=0.05)
    min_sample_size: Mapped[int] = mapped_column(Integer, nullable=False, default=100)
    primary_metric: Mapped[str] = mapped_column(
        String(64),
        nullable=False,
        default="conversion",
        comment="conversion | score | latency_ms | custom metric name",
    )

    control_variant_key: Mapped[str | None] = mapped_column(String(100), nullable=True)
    # Optional link to offline Experiment
    source_experiment_id: Mapped[str | None] = mapped_column(String(36), nullable=True)
    config: Mapped[dict[str, Any]] = mapped_column(JSON, nullable=False, default=dict)
    created_by: Mapped[str] = mapped_column(
        String(100), nullable=False, default="anonymous"
    )

```

```

)
started_at: Mapped[datetime | None] = mapped_column(
    DateTime(timezone=True), nullable=True
)
)
ended_at: Mapped[datetime | None] = mapped_column(
    DateTime(timezone=True), nullable=True
)
)
winner_variant_key: Mapped[str | None] = mapped_column(String(100), nullable=True)

variants: Mapped[list[ABVariant]] = relationship(
    back_populates="experiment",
    cascade="all, delete-orphan",
    order_by="ABVariant.created_at",
)
)
assignments: Mapped[list[ABAssignment]] = relationship(
    back_populates="experiment", cascade="all, delete-orphan"
)
)
events: Mapped[list[ABEvent]] = relationship(
    back_populates="experiment", cascade="all, delete-orphan"
)
)

class ABVariant(PKMixin, TimestampMixin, Base):
    """Traffic arm with payload (agent_config / feature flags)."""

    __tablename__ = "ab_variants"
    __table_args__ = (
        UniqueConstraint("experiment_id", "key", name="uq_ab_variant_key"),
        Index("ix_ab_variants_experiment_id", "experiment_id"),
    )

    experiment_id: Mapped[str] = mapped_column(
        String(36),
        ForeignKey("ab_experiments.id", ondelete="CASCADE"),
        nullable=False,
    )
    )
    key: Mapped[str] = mapped_column(String(100), nullable=False)
    name: Mapped[str] = mapped_column(String(255), nullable=False, default="")
    weight: Mapped[float] = mapped_column(Float, nullable=False, default=1.0)
    is_control: Mapped[bool] = mapped_column(Boolean, nullable=False, default=False)
    # Agent / feature payload applied when unit is assigned
    payload: Mapped[dict[str, Any]] = mapped_column(JSON, nullable=False, default=dict)
    description: Mapped[str] = mapped_column(Text, nullable=False, default="")

    experiment: Mapped[ABExperiment] = relationship(back_populates="variants")

class ABAssignment(PKMixin, TimestampMixin, Base):
    """Sticky unit → variant mapping."""

    __tablename__ = "ab_assignments"
    __table_args__ = (
        UniqueConstraint("experiment_id", "unit_id", name="uq_ab_assignment_unit"),
        Index("ix_ab_assignments_experiment_id", "experiment_id"),
        Index("ix_ab_assignments_variant_key", "variant_key"),
    )
    )

    experiment_id: Mapped[str] = mapped_column(
        String(36),
        ForeignKey("ab_experiments.id", ondelete="CASCADE"),
        nullable=False,
    )
    )
    unit_id: Mapped[str] = mapped_column(
        String(128),
        nullable=False,
        comment="User/session/request unit for stickiness",
    )
    )
    variant_key: Mapped[str] = mapped_column(String(100), nullable=False)
    bucket: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    context: Mapped[dict[str, Any] | None] = mapped_column(JSON, nullable=True)

```



```

experiment: Mapped[ABExperiment] = relationship(back_populates="assignments")

class ABEvent(PKMixin, TimestampMixin, Base):
    """Exposure / conversion / metric events for analysis."""

    __tablename__ = "ab_events"
    __table_args__ = (
        Index("ix_ab_events_experiment_id", "experiment_id"),
        Index("ix_ab_events_variant_key", "variant_key"),
        Index("ix_ab_events_event_type", "event_type"),
        Index("ix_ab_events_unit_id", "unit_id"),
        Index(
            "ix_ab_events_exp_type_created", "experiment_id", "event_type", "created_at"
        ),
    )

    experiment_id: Mapped[str] = mapped_column(
        String(36),
        ForeignKey("ab_experiments.id", ondelete="CASCADE"),
        nullable=False,
    )
    unit_id: Mapped[str] = mapped_column(String(128), nullable=False)
    variant_key: Mapped[str] = mapped_column(String(100), nullable=False)
    event_type: Mapped[str] = mapped_column(
        String(64),
        nullable=False,
        comment="exposure | conversion | metric",
    )
    metric_name: Mapped[str | None] = mapped_column(String(64), nullable=True)
    metric_value: Mapped[float | None] = mapped_column(Float, nullable=True)
    properties: Mapped[dict[str, Any] | None] = mapped_column(JSON, nullable=True)

    experiment: Mapped[ABExperiment] = relationship(back_populates="events")
# === File: app/models/agent_log.py ===
# AgentFlow-Eval V1.0.0
"""Agent observability log events (AOLS Phase 4 durable sink)."""

from sqlalchemy import Index, JSON, String
from sqlalchemy.orm import Mapped, mapped_column

from app.models.base import Base, PKMixin, TimestampMixin

class AgentLog(PKMixin, TimestampMixin, Base):
    """Structured runtime log row for query / statistics APIs.

    ``payload`` holds the full event envelope (tokens, latency, agent_context, ...).
    """

    __tablename__ = "agent_logs"
    __table_args__ = (
        Index("ix_agent_logs_created_at", "created_at"),
        Index("ix_agent_logs_event_created", "event", "created_at"),
        Index("ix_agent_logs_level_created", "level", "created_at"),
        Index("ix_agent_logs_task_created", "task_id", "created_at"),
        Index("ix_agent_logs_trace_created", "trace_id", "created_at"),
        Index("ix_agent_logs_service", "service"),
    )

    level: Mapped[str] = mapped_column(
        String(16),
        nullable=False,
        default="info",
        comment="info|warning|error|debug",
    )
    event: Mapped[str] = mapped_column(
        String(96),
        nullable=False,

```

```

        index=True,
        comment="dot-notation event name",
    )
    service: Mapped[str] = mapped_column(
        String(64),
        nullable=False,
        default="agentflow-api",
        comment="emitting service",
    )
    trace_id: Mapped[str | None] = mapped_column(
        String(64),
        nullable=True,
        default=None,
        comment="correlation / request id",
    )
    task_id: Mapped[str | None] = mapped_column(
        String(64),
        nullable=True,
        default=None,
        comment="evaluation task id if any",
    )
    payload: Mapped[dict] = mapped_column(
        JSON,
        nullable=False,
        default=dict,
        comment="full event fields (redacted)",
    )

    def __repr__(self) -> str:
        return f"<AgentLog {self.event} level={self.level}>"
# === File: app/models/audit_log.py ===
# AgentFlow-Eval V1.0.0
"""Audit log model for security-relevant actions."""

from sqlalchemy import Index, JSON, String, Text
from sqlalchemy.orm import Mapped, mapped_column

from app.models.base import Base, PKMixin, TenantMixin, TimestampMixin

class AuditLog(PKMixin, TenantMixin, TimestampMixin, Base):
    """Immutable-style audit record (no update API)."""

    __tablename__ = "audit_logs"
    __table_args__ = (
        Index("ix_audit_logs_created_at", "created_at"),
        Index("ix_audit_logs_action", "action"),
        Index("ix_audit_logs_resource", "resource_type", "resource_id"),
        Index("ix_audit_logs_actor_created", "actor", "created_at"),
        Index("ix_audit_logs_tenant_created", "tenant_id", "created_at"),
    )

    actor: Mapped[str] = mapped_column(
        String(100),
        nullable=False,
        default="anonymous",
        comment="操作者标识",
    )
    action: Mapped[str] = mapped_column(
        String(64),
        nullable=False,
        comment="动作, 如 task.create / task.execute",
    )
    resource_type: Mapped[str] = mapped_column(
        String(64),
        nullable=False,
        comment="资源类型",
    )
    resource_id: Mapped[str | None] = mapped_column(
        String(64),

```

```

        nullable=True,
        default=None,
        comment="资源 ID",
    )
    detail: Mapped[dict | None] = mapped_column(
        JSON,
        nullable=True,
        default=None,
        comment="附加详情",
    )
    request_id: Mapped[str | None] = mapped_column(
        String(64),
        nullable=True,
        default=None,
        comment="请求追踪 ID",
    )
    ip_address: Mapped[str | None] = mapped_column(
        String(64),
        nullable=True,
        default=None,
        comment="客户端 IP",
    )
    note: Mapped[str] = mapped_column(
        Text,
        nullable=False,
        default="",
        comment="备注",
    )

    def __repr__(self) -> str:
        return f"<AuditLog {self.action} {self.resource_type}/{self.resource_id}>"
# === File: app/models/base.py ===
# AgentFlow-Eval V1.0.0
"""基础 ORM 模型类，提供 UUID 主键与时间戳字段的公共基类。"""

from datetime import datetime
from uuid import uuid4

from sqlalchemy import DateTime, String, func
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column

class Base(DeclarativeBase):
    """所有 ORM 模型的抽象基类。"""

    pass

class TimestampMixin:
    """混入类，自动添加 created_at / updated_at 时间戳字段。

    所有业务模型通过多重继承获得时间戳能力。
    """

    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        server_default=func.now(),
        comment="记录创建时间",
    )
    updated_at: Mapped[datetime | None] = mapped_column(
        DateTime(timezone=True),
        onupdate=func.now(),
        nullable=True,
        comment="记录最后更新时间",
    )

class PKMixin:
    """混入类，提供 UUID 字符串主键。

```

兼容 SQLite（开发）和 PostgreSQL（生产）两种后端。

"""

```
id: Mapped[str] = mapped_column(
    String(36),
    primary_key=True,
    default=lambda: str(uuid4()),
    comment="UUID 主键",
)
```

```
class TenantMixin:
```

"""Optional enterprise tenant scope (nullable for Lite / pre-migration rows)."""

```
tenant_id: Mapped[str | None] = mapped_column(
    String(36),
    nullable=True,
    index=True,
    default=None,
    comment="所属租户 ID（企业多租户）；空=未绑定/单租户模式",
)
```

```
# == File: app/models/benchmark.py ==
```

```
# AgentFlow-Eval V1.0.0
```

"""Industry benchmark suite models."""

```
from __future__ import annotations
```

```
from datetime import datetime
```

```
from sqlalchemy import DateTime, Float, ForeignKey, Index, Integer, JSON, String, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship
```

```
from app.models.base import Base, PKMixin, TenantMixin, TimestampMixin
```

```
class Benchmark(PKMixin, TenantMixin, TimestampMixin, Base):
```

"""Named benchmark suite (collection of cases)."""

```
__tablename__ = "benchmarks"
__table_args__ = (
    Index("ix_benchmarks_created_by", "created_by"),
    Index("ix_benchmarks_status", "status"),
    Index("ix_benchmarks_tenant_created", "tenant_id", "created_at"),
)
```

```
name: Mapped[str] = mapped_column(String(255), nullable=False)
description: Mapped[str] = mapped_column(Text, nullable=False, default="")
status: Mapped[str] = mapped_column(
    String(32), nullable=False, default="active"
) # active|archived
created_by: Mapped[str] = mapped_column(
    String(100), nullable=False, default="anonymous"
)
tags: Mapped[list] = mapped_column(JSON, nullable=False, default=list)
meta: Mapped[dict] = mapped_column(JSON, nullable=False, default=dict)
```

```
cases: Mapped[list[BenchmarkCase]] = relationship(
    back_populates="benchmark",
    cascade="all, delete-orphan",
    order_by="BenchmarkCase.created_at",
)
```

```
runs: Mapped[list[BenchmarkRun]] = relationship(
    back_populates="benchmark",
    cascade="all, delete-orphan",
    order_by="BenchmarkRun.created_at",
)
```

```
class BenchmarkCase(PKMixin, TenantMixin, TimestampMixin, Base):
```

"""Single evaluation case inside a benchmark."""

```

__tablename__ = "benchmark_cases"
__table_args__ = (
    Index("ix_benchmark_cases_benchmark", "benchmark_id"),
    Index("ix_benchmark_cases_tenant", "tenant_id"),
)

benchmark_id: Mapped[str] = mapped_column(
    String(36),
    ForeignKey("benchmarks.id", ondelete="CASCADE"),
    nullable=False,
)

name: Mapped[str] = mapped_column(String(255), nullable=False, default="")
user_query: Mapped[str] = mapped_column(Text, nullable=False)
expected_output: Mapped[str] = mapped_column(Text, nullable=False, default="")
expected_tools: Mapped[list] = mapped_column(JSON, nullable=False, default=list)
weight: Mapped[float] = mapped_column(Float, nullable=False, default=1.0)
extra_metadata: Mapped[dict] = mapped_column(JSON, nullable=False, default=dict)

benchmark: Mapped[Benchmark] = relationship(back_populates="cases")

class BenchmarkRun(PKMixin, TenantMixin, TimestampMixin, Base):
    """One execution of a benchmark against an agent configuration."""

    __tablename__ = "benchmark_runs"
    __table_args__ = (
        Index("ix_benchmark_runs_benchmark", "benchmark_id"),
        Index("ix_benchmark_runs_task", "task_id"),
        Index("ix_benchmark_runs_status", "status"),
        Index("ix_benchmark_runs_tenant", "tenant_id"),
    )

    benchmark_id: Mapped[str] = mapped_column(
        String(36),
        ForeignKey("benchmarks.id", ondelete="CASCADE"),
        nullable=False,
    )

    task_id: Mapped[str | None] = mapped_column(String(36), nullable=True, index=True)
    label: Mapped[str] = mapped_column(
        String(128), nullable=False, default="default"
    )
    # leaderboard series key
    agent_config: Mapped[dict] = mapped_column(JSON, nullable=False, default=dict)
    status: Mapped[str] = mapped_column(
        String(32), nullable=False, default="pending"
    )
    # pending|queued|running|completed|failed
    created_by: Mapped[str] = mapped_column(
        String(100), nullable=False, default="anonymous"
    )

    started_at: Mapped[datetime | None] = mapped_column(
        DateTime(timezone=True), nullable=True
    )

    finished_at: Mapped[datetime | None] = mapped_column(
        DateTime(timezone=True), nullable=True
    )

    summary: Mapped[dict] = mapped_column(JSON, nullable=False, default=dict)

    benchmark: Mapped[Benchmark] = relationship(back_populates="runs")
    results: Mapped[list[BenchmarkResult]] = relationship(
        back_populates="run",
        cascade="all, delete-orphan",
    )

class BenchmarkResult(PKMixin, TenantMixin, TimestampMixin, Base):
    """Per-case metrics for a benchmark run."""

    __tablename__ = "benchmark_results"
    __table_args__ = (
        Index("ix_benchmark_results_run", "run_id"),

```

```

        Index("ix_benchmark_results_case", "case_id"),
        Index("ix_benchmark_results_tenant", "tenant_id"),
    )

    run_id: Mapped[str] = mapped_column(
        String(36),
        ForeignKey("benchmark_runs.id", ondelete="CASCADE"),
        nullable=False,
    )
    case_id: Mapped[str | None] = mapped_column(String(36), nullable=True)
    trace_id: Mapped[str | None] = mapped_column(String(36), nullable=True)
    accuracy: Mapped[float | None] = mapped_column(Float, nullable=True)
    quality: Mapped[float | None] = mapped_column(Float, nullable=True)
    latency_ms: Mapped[float | None] = mapped_column(Float, nullable=True)
    cost: Mapped[float | None] = mapped_column(Float, nullable=True)
    tokens: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    score: Mapped[float | None] = mapped_column(Float, nullable=True)
    detail: Mapped[dict] = mapped_column(JSON, nullable=False, default=dict)

    run: Mapped[BenchmarkRun] = relationship(back_populates="results")
# === File: app/models/billing.py ===
# AgentFlow-Eval V1.0.0
"""SaaS billing models: plans, subscriptions, usage, quotas, invoices."""

from __future__ import annotations

from datetime import datetime

from sqlalchemy import (
    Boolean,
    DateTime,
    Float,
    Index,
    Integer,
    JSON,
    String,
    Text,
    UniqueConstraint,
    func,
)
from sqlalchemy.orm import Mapped, mapped_column

from app.models.base import Base, PKMixin, TenantMixin, TimestampMixin

class BillingPlan(PKMixin, TimestampMixin, Base):
    """Subscription plan catalog."""

    __tablename__ = "billing_plans"
    __table_args__ = (UniqueConstraint("code", name="uq_billing_plans_code"),)

    code: Mapped[str] = mapped_column(String(64), nullable=False, index=True)
    name: Mapped[str] = mapped_column(String(128), nullable=False)
    description: Mapped[str] = mapped_column(Text, nullable=False, default="")
    price_month_cents: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    billing_cycle: Mapped[str] = mapped_column(
        String(32), nullable=False, default="monthly"
    )
    # monthly | yearly
    token_quota: Mapped[int] = mapped_column(Integer, nullable=False, default=100_000)
    task_quota: Mapped[int] = mapped_column(Integer, nullable=False, default=100)
    storage_quota_mb: Mapped[int] = mapped_column(Integer, nullable=False, default=1024)
    plugin_quota: Mapped[int] = mapped_column(Integer, nullable=False, default=10)
    # Free-form feature flags, e.g. {"plugins": ["echo_tool"], "vision": true}
    features: Mapped[dict] = mapped_column(JSON, nullable=False, default=dict)
    is_public: Mapped[bool] = mapped_column(Boolean, nullable=False, default=True)
    is_active: Mapped[bool] = mapped_column(Boolean, nullable=False, default=True)

class Subscription(PKMixin, TenantMixin, TimestampMixin, Base):
    """Actor/tenant subscription to a plan."""

```

```

__tablename__ = "subscriptions"
__table_args__ = (
    Index("ix_subscriptions_actor_status", "actor", "status"),
    Index("ix_subscriptions_tenant_status", "tenant_id", "status"),
)

actor: Mapped[str] = mapped_column(String(100), nullable=False, index=True)
plan_id: Mapped[str] = mapped_column(String(36), nullable=False, index=True)
status: Mapped[str] = mapped_column(
    String(32), nullable=False, default="active"
) # active|past_due|canceled|trialing
period_start: Mapped[datetime | None] = mapped_column(
    DateTime(timezone=True), nullable=True
)
period_end: Mapped[datetime | None] = mapped_column(
    DateTime(timezone=True), nullable=True
)
external_ref: Mapped[str | None] = mapped_column(String(128), nullable=True)

class UsageRecord(PKMixin, TenantMixin, Base):
    """Immutable usage event (token / task / judge / storage)."""

    __tablename__ = "usage_records"
    __table_args__ = (
        Index("ix_usage_actor_created", "actor", "created_at"),
        Index("ix_usage_metric_created", "metric", "created_at"),
        Index("ix_usage_trace_id", "trace_id"),
        Index("ix_usage_tenant_created", "tenant_id", "created_at"),
    )

    actor: Mapped[str] = mapped_column(String(100), nullable=False, index=True)
    metric: Mapped[str] = mapped_column(
        String(32), nullable=False
    ) # token|task|judge|storage
    quantity: Mapped[float] = mapped_column(Float, nullable=False, default=1.0)
    unit_cost: Mapped[float] = mapped_column(Float, nullable=False, default=0.0)
    ref_type: Mapped[str | None] = mapped_column(String(64), nullable=True)
    ref_id: Mapped[str | None] = mapped_column(String(64), nullable=True)
    trace_id: Mapped[str | None] = mapped_column(String(64), nullable=True)
    extra: Mapped[dict] = mapped_column(JSON, nullable=False, default=dict)
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        server_default=func.now(),
        nullable=False,
    )

class QuotaBalance(PKMixin, TimestampMixin, Base):
    """Period-scoped counters per actor."""

    __tablename__ = "quota_balances"
    __table_args__ = (
        UniqueConstraint("actor", "period", name="uq_quota_actor_period"),
        Index("ix_quota_actor_period", "actor", "period"),
    )

    actor: Mapped[str] = mapped_column(String(100), nullable=False)
    # YYYY-MM
    period: Mapped[str] = mapped_column(String(7), nullable=False)
    token_used: Mapped[float] = mapped_column(Float, nullable=False, default=0.0)
    token_limit: Mapped[int] = mapped_column(Integer, nullable=False, default=100_000)
    task_used: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    task_limit: Mapped[int] = mapped_column(Integer, nullable=False, default=100)
    storage_used_mb: Mapped[float] = mapped_column(Float, nullable=False, default=0.0)
    storage_limit_mb: Mapped[int] = mapped_column(Integer, nullable=False, default=1024)
    plugin_used: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    plugin_limit: Mapped[int] = mapped_column(Integer, nullable=False, default=10)

```

```

class Invoice(PKMixin, TenantMixin, TimestampMixin, Base):
    """Billing period invoice (draft until payment integration)."""

    __tablename__ = "invoices"
    __table_args__ = (
        Index("ix_invoices_actor_period", "actor", "period"),
        Index("ix_invoices_tenant_period", "tenant_id", "period"),
    )

    actor: Mapped[str] = mapped_column(String(100), nullable=False, index=True)
    period: Mapped[str] = mapped_column(String(7), nullable=False)
    amount_cents: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    status: Mapped[str] = mapped_column(
        String(32), nullable=False, default="draft"
    ) # draft|open|paid|void
    line_items: Mapped[list] = mapped_column(JSON, nullable=False, default=list)
    issued_at: Mapped[datetime | None] = mapped_column(
        DateTime(timezone=True), nullable=True
    )

# === File: app/models/experiment.py ===
# AgentFlow-Eval V1.0.0
"""Experiment models — multi-run comparison on the same test suites."""

from __future__ import annotations

from typing import TYPE_CHECKING, Any

from sqlalchemy import ForeignKey, Index, JSON, String, Text, UniqueConstraint
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.models.base import Base, PKMixin, TenantMixin, TimestampMixin

if TYPE_CHECKING:
    from app.models.task import Task

class Experiment(PKMixin, TenantMixin, TimestampMixin, Base):
    """A comparison experiment grouping multiple evaluation runs.

    Suites are snapshotted as JSON so later base-task edits do not alter
    historical comparisons. Each run materializes a Task with cloned suites.
    """

    __tablename__ = "experiments"
    __table_args__ = (
        Index("ix_experiments_created_by", "created_by"),
        Index("ix_experiments_created_at", "created_at"),
        Index("ix_experiments_base_task_id", "base_task_id"),
        Index("ix_experiments_owner_created", "created_by", "created_at"),
        Index("ix_experiments_tenant_created", "tenant_id", "created_at"),
    )

    name: Mapped[str] = mapped_column(String(255), nullable=False, comment="实验名称")
    description: Mapped[str] = mapped_column(
        Text, nullable=False, default="", comment="实验描述"
    )
    base_task_id: Mapped[str | None] = mapped_column(
        String(36),
        nullable=True,
        comment="可选：套件来源任务 ID",
    )
    suite_snapshot: Mapped[list[dict[str, Any]]] = mapped_column(
        JSON,
        nullable=False,
        default=list,
        comment="测试用例快照 [{user_query, expected_output, expected_tools}]",
    )
    created_by: Mapped[str] = mapped_column(
        String(100),

```



```

        nullable=False,
        default="anonymous",
        comment="创建者 actor",
    )

    runs: Mapped[list[ExperimentRun]] = relationship(
        back_populates="experiment",
        cascade="all, delete-orphan",
        order_by="ExperimentRun.created_at",
    )

    def __repr__(self) -> str:
        return f"<Experiment id={self.id} name={self.name!r}>"

class ExperimentRun(PKMixin, TenantMixin, TimestampMixin, Base):
    """One evaluation run inside an experiment (maps 1:1 to a Task)."""

    __tablename__ = "experiment_runs"
    __table_args__ = (
        UniqueConstraint("experiment_id", "label", name="uq_experiment_run_label"),
        Index("ix_experiment_runs_tenant", "tenant_id"),
    )

    experiment_id: Mapped[str] = mapped_column(
        String(36),
        ForeignKey("experiments.id", ondelete="CASCADE"),
        nullable=False,
        index=True,
        comment="所属实验",
    )

    task_id: Mapped[str] = mapped_column(
        String(36),
        ForeignKey("tasks.id", ondelete="CASCADE"),
        nullable=False,
        index=True,
        comment="对应评测任务",
    )

    label: Mapped[str] = mapped_column(
        String(100),
        nullable=False,
        comment="变体标签, 如 gpt-4o / http-agent-v1",
    )

    agent_config: Mapped[dict[str, Any]] = mapped_column(
        JSON,
        nullable=False,
        default=dict,
        comment="该 run 使用的 agent_config 快照",
    )

    experiment: Mapped[Experiment] = relationship(back_populates="runs")
    task: Mapped[Task] = relationship()

    def __repr__(self) -> str:
        return f"<ExperimentRun id={self.id} label={self.label!r} task={self.task_id!r}>"
# === File: app/models/media_asset.py ===
# AgentFlow-Eval V1.0.0
"""MediaAsset — uploaded multimodal files linked to tasks/suites."""

from __future__ import annotations

from typing import TYPE_CHECKING, Any

from sqlalchemy import ForeignKey, Index, Integer, JSON, String, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.models.base import Base, PKMixin, TenantMixin, TimestampMixin

if TYPE_CHECKING:
    from app.models.task import Task

```

```

from app.models.test_suite import TestSuite

class MediaAsset(PKMixin, TenantMixin, TimestampMixin, Base):
    """Stored multimodal artifact with extraction results."""

    __tablename__ = "media_assets"
    __table_args__ = (
        Index("ix_media_assets_task_id", "task_id"),
        Index("ix_media_assets_suite_id", "test_suite_id"),
        Index("ix_media_assets_created_by", "created_by"),
        Index("ix_media_assets_kind", "media_kind"),
        Index("ix_media_assets_tenant", "tenant_id"),
    )

    filename: Mapped[str] = mapped_column(String(512), nullable=False)
    content_type: Mapped[str] = mapped_column(
        String(128), nullable=False, default="application/octet-stream"
    )
    size_bytes: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    sha256: Mapped[str] = mapped_column(String(64), nullable=False, default="")
    media_kind: Mapped[str] = mapped_column(String(32), nullable=False, default="other")

    storage_backend: Mapped[str] = mapped_column(
        String(32), nullable=False, default="local"
    )
    storage_key: Mapped[str] = mapped_column(String(1024), nullable=False)

    extracted_text: Mapped[str] = mapped_column(Text, nullable=False, default="")
    features: Mapped[dict[str, Any] | None] = mapped_column(
        JSON, nullable=True, default=None
    )
    extract_meta: Mapped[dict[str, Any] | None] = mapped_column(
        JSON, nullable=True, default=None
    )

    task_id: Mapped[str | None] = mapped_column(
        String(36),
        ForeignKey("tasks.id", ondelete="SET NULL"),
        nullable=True,
    )
    test_suite_id: Mapped[str | None] = mapped_column(
        String(36),
        ForeignKey("test_suites.id", ondelete="SET NULL"),
        nullable=True,
    )
    created_by: Mapped[str] = mapped_column(
        String(100), nullable=False, default="anonymous"
    )

    task: Mapped[Task | None] = relationship()
    test_suite: Mapped[TestSuite | None] = relationship()

    def __repr__(self) -> str:
        return (
            f"<MediaAsset id={self.id} kind={self.media_kind!r} file={self.filename!r}>"
        )
# === File: app/models/metric_score.py ===
# AgentFlow-Eval V1.0.0
"""MetricScore model —— 单项评分指标记录。"""

from typing import TYPE_CHECKING

from sqlalchemy import Boolean, Float, ForeignKey, Index, JSON, String, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.models.base import Base, PKMixin, TenantMixin, TimestampMixin

if TYPE_CHECKING:
    from app.models.trace import Trace

```

```

class MetricScore(PKMixin, TenantMixin, TimestampMixin, Base):
    """评测指标得分。

    记录每个 Trace 在某个指标维度（如准确率、工具调用正确性等）的评分，
    支持 LLM 自动评分与人工审核覆盖。
    """

    __tablename__ = "metric_scores"
    __table_args__ = (
        Index("ix_metric_scores_trace_metric", "trace_id", "metric_name"),
        Index("ix_metric_scores_human_reviewed", "is_human_reviewed"),
        Index("ix_metric_scores_tenant", "tenant_id"),
    )

    trace_id: Mapped[str] = mapped_column(
        ForeignKey("traces.id", ondelete="CASCADE"),
        nullable=False,
        index=True,
        comment="FK to trace",
    )
    metric_name: Mapped[str] = mapped_column(
        String(100),
        nullable=False,
        comment="metric name",
    )
    score: Mapped[float] = mapped_column(
        Float,
        nullable=False,
        comment="score 0-100",
    )
    reason: Mapped[str] = mapped_column(
        Text,
        nullable=False,
        default="",
        comment="deduction reason",
    )
    extra_metadata: Mapped[dict | None] = mapped_column(
        JSON,
        nullable=True,
        default=None,
        comment="extra metadata",
    )

    # ---- v1.0 新增: LLM 评分置信度 ----
    confidence: Mapped[float | None] = mapped_column(
        Float,
        nullable=True,
        default=None,
        comment="LLM 评分置信度 0.0 ~ 1.0",
    )

    # ---- v1.0 新增: 人工审核覆盖 ----
    is_human_reviewed: Mapped[bool] = mapped_column(
        Boolean,
        nullable=False,
        default=False,
        comment="是否经过人工审核",
    )
    human_score: Mapped[float | None] = mapped_column(
        Float,
        nullable=True,
        default=None,
        comment="人工审核给出的分数（覆盖 LLM 评分）",
    )
    reviewer: Mapped[str | None] = mapped_column(
        String(100),
        nullable=True,
        default=None,
    )

```

```

        comment="人工审核人员标识",
    )

# ---- 关系 ----
trace: Mapped["Trace"] = relationship(back_populates="metric_scores")

@property
def effective_score(self) -> float:
    """返回最终有效分数：优先使用人工审核分数。"""
    if self.is_human_reviewed and self.human_score is not None:
        return self.human_score
    return self.score

def __repr__(self) -> str:
    return f"<MetricScore name={self.metric_name!r} score={self.score} reviewed={self.is_human_reviewed}>"
# === File: app/models/slow_task.py ===
# AgentFlow-Eval V1.0.0
"""Persisted slow-task samples for diagnostics across restarts."""

from __future__ import annotations

from datetime import datetime

from sqlalchemy import DateTime, Float, Index, JSON, String, func
from sqlalchemy.orm import Mapped, mapped_column

from app.models.base import Base, PKMixin

class SlowTaskEvent(PKMixin, Base):
    """One slow agent/judge/pipeline sample above configured threshold."""

    __tablename__ = "slow_task_events"
    __table_args__ = (
        Index("ix_slow_task_created", "created_at"),
        Index("ix_slow_task_stage_created", "stage", "created_at"),
        Index("ix_slow_task_trace", "trace_id"),
        Index("ix_slow_task_actor", "actor"),
    )

    stage: Mapped[str] = mapped_column(String(32), nullable=False, index=True)
    duration_sec: Mapped[float] = mapped_column(Float, nullable=False)
    threshold_sec: Mapped[float] = mapped_column(Float, nullable=False)
    ref_id: Mapped[str | None] = mapped_column(String(64), nullable=True)
    status: Mapped[str] = mapped_column(String(32), nullable=False, default="ok")
    trace_id: Mapped[str | None] = mapped_column(String(64), nullable=True)
    actor: Mapped[str | None] = mapped_column(String(100), nullable=True)
    extra: Mapped[dict] = mapped_column(JSON, nullable=False, default=dict)
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        server_default=func.now(),
        nullable=False,
    )
# === File: app/models/task.py ===
# AgentFlow-Eval V1.0.0
"""Task 任务模型 —— 表示一次评测任务的顶层聚合根。"""

import enum
from typing import TYPE_CHECKING

from sqlalchemy import Boolean, Index, JSON, Enum, String, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.models.base import Base, PKMixin, TenantMixin, TimestampMixin

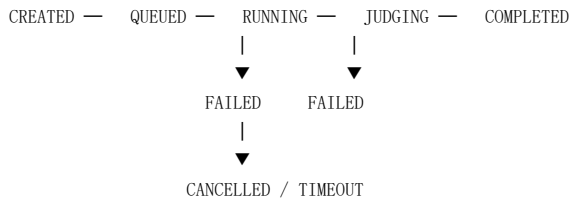
if TYPE_CHECKING:
    from app.models.test_suite import TestSuite

class TaskStatus(str, enum.Enum):

```

"""任务状态枚举 —— 生产级状态机。

状态流转：



"""

```

CREATED = "created"
QUEUED = "queued"
RUNNING = "running"
WAITING_TOOL = "waiting_tool"
JUDGING = "judging"
COMPLETED = "completed"
FAILED = "failed"
CANCELLED = "cancelled"
TIMEOUT = "timeout"
  
```

---- 合法流转矩阵 ----

@classmethod

def allowed_transitions(cls) -> dict["TaskStatus", set["TaskStatus"]]:

"""返回每个状态允许流转的目标状态集合。"""

```

    return {
        cls.CREATED: {cls.QUEUED, cls.CANCELLED},
        cls.QUEUED: {cls.RUNNING, cls.CANCELLED, cls.TIMEOUT},
        cls.RUNNING: {
            cls.WAITING_TOOL,
            cls.JUDGING,
            cls.FAILED,
            cls.CANCELLED,
            cls.TIMEOUT,
        },
        cls.WAITING_TOOL: {cls.RUNNING, cls.FAILED, cls.CANCELLED, cls.TIMEOUT},
        cls.JUDGING: {cls.COMPLETED, cls.FAILED, cls.CANCELLED},
        cls.COMPLETED: set(),
        cls.FAILED: set(),
        cls.CANCELLED: set(),
        cls.TIMEOUT: set(),
    }
  
```

def can_transition_to(self, target: "TaskStatus") -> bool:

"""检查是否允许从当前状态流转到目标状态。"""

return target in self.allowed_transitions().get(self, set())

【系统设计决策】2026-06-10：任务状态迁移（如从 RUNNING → COMPLETED）

在并发场景下（如用户手动取消的同时，Celery Worker 刚好执行完毕），

会出现竞态条件。初期方案使用应用层的状态机校验（can_transition_to），

但在压力测试中发现，两个请求几乎同时到达时，状态检查通过但实际更新

时状态已变。最终我们采用数据库层的乐观锁策略：

UPDATE tasks SET status='completed' WHERE id=:id AND status='running'

如果影响行数为 0，则抛出 TaskStateError，由调用方重试或忽略。

该方案是权衡了并发性能与数据一致性后的工程妥协。

class Task(PKMixin, TenantMixin, TimestampMixin, Base):

"""评测任务。

一个 Task 包含多个 TestSuite 测试用例，运行时逐条运行并汇总评分。

"""

__tablename__ = "tasks"

__table_args__ = (

List: WHERE created_by=? AND is_archived=0 ORDER BY created_at DESC

Index(

"ix_tasks_owner_archived_created",

"created_by",

"is_archived",

```

        "created_at",
    ),
    # Dashboard: WHERE status=? ORDER BY created_at DESC
    Index("ix_tasks_status_created", "status", "created_at"),
    Index("ix_tasks_created_by", "created_by"),
    Index("ix_tasks_celery_task_id", "celery_task_id"),
    Index("ix_tasks_is_archived", "is_archived"),
    Index("ix_tasks_tenant_created", "tenant_id", "created_at"),
)

name: Mapped[str] = mapped_column(
    String(255),
    nullable=False,
    comment="任务名称",
)

description: Mapped[str] = mapped_column(
    Text,
    nullable=False,
    default="",
    comment="任务描述",
)

status: Mapped[TaskStatus] = mapped_column(
    Enum(
        TaskStatus,
        name="task_status",
        native_enum=False,
        length=20,
        values_callable=lambda enum_cls: [e.value for e in enum_cls],
    ),
    nullable=False,
    default=TaskStatus.CREATED,
    comment="任务状态",
)

agent_config: Mapped[dict] = mapped_column(
    JSON,
    nullable=False,
    default=dict,
    comment="Agent 配置参数 (JSON 对象)",
)

celery_task_id: Mapped[str | None] = mapped_column(
    String(255),
    nullable=True,
    default=None,
    comment="Celery 异步任务 ID, 用于取消和状态追踪",
)

is_archived: Mapped[bool] = mapped_column(
    Boolean,
    nullable=False,
    default=False,
    comment="是否已归档 (软归档, 列表默认隐藏)",
)

created_by: Mapped[str] = mapped_column(
    String(100),
    nullable=False,
    default="anonymous",
    comment="创建者 actor (API Key 映射名), 用于轻量多租户隔离",
)

# ---- 关系 ----
test_suites: Mapped[list["TestSuite"]] = relationship(
    back_populates="task",
    cascade="all, delete-orphan",
)

def __repr__(self) -> str:
    return f"<Task id={self.id} name={self.name!r} status={self.status.value}>"
# === File: app/models/tenant.py ===
# AgentFlow-Eval V1.0.0
"""Enterprise multi-tenant models: tenants + membership."""

```

```

text = str(raw).strip()
if not text:
    return []
if text.startswith("["):
    try:
        data = json.loads(text)
        if isinstance(data, list):
            return [str(x).strip() for x in data if str(x).strip()]
    except json.JSONDecodeError:
        pass
return [t.strip() for t in text.split("|") if t.strip()] or [
    t.strip() for t in text.split(",") if t.strip()
]

def _suites_from_items(task_id: str, items: list[dict]) -> list[TestSuite]:
    suites: list[TestSuite] = []
    for item in items:
        if not item.get("user_query"):
            continue
        suites.append(
            TestSuite(
                task_id=task_id,
                user_query=str(item.get("user_query", "")),
                expected_output=str(item.get("expected_output", "")),
                expected_tools=_parse_expected_tools(item.get("expected_tools")),
                extra_metadata=item.get("extra_metadata"),
            )
        )
    return suites

@router.get("", response_model=TaskListResponse)
@require_permission(Permission.TASK_READ)
async def list_tasks(
    request: Request,
    page: int = Query(1, ge=1, description="Page number"),
    page_size: int = Query(20, ge=1, le=100, description="Items per page"),
    status: str | None = Query(None, description="Filter by status"),
    include_archived: bool = Query(False, description="Include archived tasks"),
    session: AsyncSession = Depends(get_db),
) -> Any:
    """Get paginated list of evaluation tasks (scoped by actor when tenancy on).

    Cached 30s with version-based invalidation on task mutations.
    """
    from app.core.cache.services import get_cached_task_list
    from app.core.tenant_context import extract_tenant_header, resolve_tenant_context
    from app.schemas.task import TaskResponse as TR

    actor = _actor(request)
    role = _role(request)
    await resolve_tenant_context(
        session,
        actor=actor,
        header_value=extract_tenant_header(request),
        system_role=role,
    )
    payload = await get_cached_task_list(
        session,
        actor=actor,
        role=role,
        page=page,
        page_size=page_size,
        status=status,
        include_archived=include_archived,
    )
    items = [TR(**item) for item in payload["items"]]
    return TaskListResponse(
        items=items,

```

```

        total=payload["total"],
        page=payload["page"],
        page_size=payload["page_size"],
    )

@router.post("", response_model=TaskResponse, status_code=201)
@require_permission(Permission.TASK_CREATE)
async def create_task(
    body: TaskCreate,
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> Any:
    """Create a new evaluation task owned by the current actor."""
    actor = _actor(request)
    from app.core.tenant_context import (
        current_tenant_id,
        extract_tenant_header,
        resolve_tenant_context,
    )

    role = getattr(request.state, "role", None)
    role_s = role.value if hasattr(role, "value") else str(role or "")
    await resolve_tenant_context(
        session,
        actor=actor,
        header_value=extract_tenant_header(request),
        system_role=role_s,
    )
    tenant_id = current_tenant_id()

    task = Task(
        name=body.name,
        description=body.description,
        agent_config=body.agent_config,
        status=TaskStatus.CREATED,
        created_by=actor,
        tenant_id=tenant_id,
    )
    session.add(task)
    await session.flush()
    await write_audit(
        session,
        action="task.create",
        resource_type="task",
        resource_id=task.id,
        actor=actor,
        detail={"name": task.name, "created_by": actor},
        request_id=_request_id(request),
        ip=_client_ip(request),
    )
    await session.commit()
    await session.refresh(task)
    try:
        from app.core.observability.metrics import observe_task_created

        observe_task_created(tenant=actor, agent_config=task.agent_config or {})
    except Exception:
        pass
    try:
        from app.core.cache.invalidation import (
            invalidate_dashboard,
            invalidate_task_lists,
        )

        from app.core.cache.services import set_cached_task_detail

        # Write-through detail + bump list/dashboard versions
        await set_cached_task_detail(task, suite_count=0)
        await invalidate_task_lists(actor)
        await invalidate_dashboard(actor)

```



```

except Exception:
    pass
return await _task_to_response(session, task)

@router.get("/{task_id}", response_model=TaskResponse)
@require_permission(Permission.TASK_READ)
async def get_task(
    task_id: str,
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> Any:
    """Get task details by ID (owner-scoped when tenancy on). Cached 5 min."""
    from app.core.cache.services import get_cached_task_detail

    task = await _load_task(session, task_id, _actor(request), role=_role(request))
    payload = await get_cached_task_detail(session, task)
    return TaskResponse(**payload)

@router.delete("/{task_id}", status_code=200)
@require_permission(Permission.TASK_DELETE)
async def delete_task(
    task_id: str,
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> dict:
    """Delete a task and its associated records."""
    actor = _actor(request)
    task = await _load_task(session, task_id, actor, role=_role(request))

    await write_audit(
        session,
        action="task.delete",
        resource_type="task",
        resource_id=task_id,
        actor=actor,
        detail={"name": task.name},
        request_id=_request_id(request),
        ip=_client_ip(request),
    )
    await session.delete(task)
    await session.commit()
    try:
        from app.core.cache.invalidation import invalidate_task

        await invalidate_task(task_id, actor=actor)
    except Exception:
        pass
    return {"message": "Task deleted", "task_id": task_id}

@router.post("/{task_id}/execute")
@require_permission(Permission.TASK_EXECUTE)
async def execute_task(
    task_id: str,
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> dict:
    """Trigger async execution of the evaluation task."""
    actor = _actor(request)
    task = await _load_task(session, task_id, actor, role=_role(request))

    if not task.status.can_transition_to(TaskStatus.QUEUED):
        raise HTTPException(
            status_code=409,
            detail=f"Cannot execute task in '{task.status.value}' state. "
            f"Only tasks in 'created' state can be executed.",
        )

```

```

# SaaS quota gate (no-op when BILLING_ENABLED=false)
try:
    from app.core.billing.service import QuotaExceededError, get_billing_service

    await get_billing_service().ensure_task_quota(session, actor)
except QuotaExceededError:
    raise
except Exception:
    pass

from app.core.profiles import get_task_queue
from app.core.observability.tracing import get_trace_id

prev_status = task.status.value
task.status = TaskStatus.QUEUED
await session.flush()

# Pluggable queue — pass TraceID into worker for full-chain correlation
queue = get_task_queue()
trace_id = get_trace_id() or _request_id(request)
enq = queue.enqueue(
    "run_full_evaluation",
    args=(task_id,),
    kwargs={"_trace_id": trace_id} if trace_id else None,
)
task.celery_task_id = enq.task_id

# Meter task execution (best-effort)
try:
    from app.core.billing.service import get_billing_service

    await get_billing_service().record_usage(
        session,
        actor=actor,
        metric="task",
        quantity=1,
        ref_type="task",
        ref_id=task_id,
        trace_id=trace_id or None,
    )
except Exception:
    pass
await write_audit(
    session,
    action="task.execute",
    resource_type="task",
    resource_id=task_id,
    actor=actor,
    detail={
        "celery_task_id": enq.task_id,
        "queue_backend": enq.backend,
        "eager": enq.eager,
        "trace_id": trace_id,
    },
    request_id=trace_id or _request_id(request),
    ip=_client_ip(request),
)
await session.commit()
await _invalidate_task_caches(task_id, actor)

try:
    from app.core.events import publish_task_status

    publish_task_status(
        task.id,
        task.name,
        TaskStatus.QUEUED.value,
        prev_status=prev_status,
        actor=actor,
    )

```

```

except Exception:
    pass

return {
    "task_id": task_id,
    "status": "queued",
    "celery_task_id": enq.task_id,
    "queue_backend": enq.backend,
    "message": "Task submitted to execution queue",
}

@router.post("/{task_id}/test-suites", status_code=201)
@require_permission(Permission.TASK_UPDATE)
async def create_test_suites(
    task_id: str,
    suites: list[dict],
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> dict:
    """Batch-create test suites for a task."""
    await _load_task(session, task_id, _actor(request), role=_role(request))

    created_suites = _suites_from_items(task_id, suites)
    for suite in created_suites:
        session.add(suite)

    await session.commit()
    await _invalidate_task_caches(task_id, _actor(request))
    return {"task_id": task_id, "created": len(created_suites)}

@router.post("/{task_id}/test-suites/upload", status_code=201)
@require_permission(Permission.TASK_UPDATE)
async def upload_test_suites(
    task_id: str,
    request: Request,
    file: UploadFile = File(..., description="CSV or JSON test suite file"),
    session: AsyncSession = Depends(get_db),
) -> dict:
    """Upload CSV/JSON file to batch-import test suites."""
    actor = _actor(request)
    await _load_task(session, task_id, actor, role=_role(request))

    filename = (file.filename or "").lower()
    raw = await file.read()
    if not raw:
        raise HTTPException(status_code=400, detail="Empty file")

    try:
        text = raw.decode("utf-8-sig")
    except UnicodeDecodeError as exc:
        raise HTTPException(
            status_code=400, detail="File must be UTF-8 encoded"
        ) from exc

    items: list[dict] = []
    if filename.endswith(".json") or text.lstrip().startswith("["):
        try:
            data = json.loads(text)
        except json.JSONDecodeError as exc:
            raise HTTPException(status_code=400, detail=f"Invalid JSON: {exc}") from exc
        if not isinstance(data, list):
            raise HTTPException(
                status_code=400, detail="JSON must be an array of test cases"
            )
        items = [x for x in data if isinstance(x, dict)]
    elif filename.endswith(".csv") or "," in text.splitlines()[0]:
        reader = csv.DictReader(io.StringIO(text))
        if not reader.fieldnames or "user_query" not in reader.fieldnames:

```

```

        raise HTTPException(
            status_code=400,
            detail="CSV must include a 'user_query' column "
            "(optional: expected_output, expected_tools)",
        )
    items = list(reader)
else:
    raise HTTPException(
        status_code=400,
        detail="Unsupported file type. Upload .csv or .json",
    )

created_suites = _suites_from_items(task_id, items)
if not created_suites:
    raise HTTPException(status_code=400, detail="No valid test cases found in file")

for suite in created_suites:
    session.add(suite)
await write_audit(
    session,
    action="task.upload_suites",
    resource_type="task",
    resource_id=task_id,
    actor=actor,
    detail={"filename": file.filename, "created": len(created_suites)},
    request_id=request_id(request),
    ip=_client_ip(request),
)
await session.commit()
await _invalidate_task_caches(task_id, actor)

return {
    "task_id": task_id,
    "created": len(created_suites),
    "filename": file.filename,
    "message": f"Imported {len(created_suites)} test suite(s)",
}

@router.post("/{task_id}/cancel")
@require_permission(Permission.TASK_CANCEL)
async def cancel_task(
    task_id: str,
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> dict:
    """Cancel a running evaluation task."""
    actor = _actor(request)
    task = await _load_task(session, task_id, actor, role=_role(request))

    if not task.status.can_transition_to(TaskStatus.CANCELLED):
        raise HTTPException(
            status_code=409,
            detail=f"Cannot cancel task in '{task.status.value}' state.",
        )

    if task.celery_task_id:
        from app.core.profiles import get_task_queue

        get_task_queue().revoke(task.celery_task_id, terminate=True)

    prev_status = task.status.value
    task.status = TaskStatus.CANCELLED
    await write_audit(
        session,
        action="task.cancel",
        resource_type="task",
        resource_id=task_id,
        actor=actor,
        request_id=request_id(request),
    )

```

```

        ip=_client_ip(request),
    )
    await session.commit()
    await _invalidate_task_caches(task_id, actor)

    try:
        from app.core.events import publish_task_status

        publish_task_status(
            task.id,
            task.name,
            TaskStatus.CANCELLED.value,
            prev_status=prev_status,
            actor=actor,
        )
    except Exception:
        pass

    return {
        "task_id": task_id,
        "status": "cancelled",
        "message": "Task has been cancelled",
    }

@router.post("/{task_id}/archive")
@require_permission(Permission.TASK_UPDATE)
async def archive_task(
    task_id: str,
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> dict:
    """Soft-archive a terminal-state task (hidden from default list)."""
    actor = _actor(request)
    task = await _load_task(session, task_id, actor, role=_role(request))

    terminal_states = {
        TaskStatus.COMPLETED,
        TaskStatus.FAILED,
        TaskStatus.CANCELLED,
        TaskStatus.TIMEOUT,
    }

    if task.status not in terminal_states:
        raise HTTPException(
            status_code=409,
            detail=f"Cannot archive task in '{task.status.value}' state. "
            f"Only terminal-state tasks can be archived.",
        )

    task.is_archived = True
    await write_audit(
        session,
        action="task.archive",
        resource_type="task",
        resource_id=task_id,
        actor=actor,
        request_id=_request_id(request),
        ip=_client_ip(request),
    )
    await session.commit()
    await _invalidate_task_caches(task_id, actor)

    return {
        "task_id": task_id,
        "status": task.status.value,
        "is_archived": True,
        "message": "Task archived successfully",
    }

```

```

@router.post("/{task_id}/unarchive")
@require_permission(Permission.TASK_UPDATE)
async def unarchive_task(
    task_id: str,
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> dict:
    """Restore an archived task to the default list."""
    actor = _actor(request)
    task = await _load_task(session, task_id, actor, role=_role(request))

    task.is_archived = False
    await session.commit()
    await _invalidate_task_caches(task_id, actor)

    return {
        "task_id": task_id,
        "status": task.status.value,
        "is_archived": False,
        "message": "Task unarchived successfully",
    }
# === File: app/api/v1/endpoints/tenants.py ===
# AgentFlow-Eval V1.0.0
"""Enterprise tenant management API."""

from __future__ import annotations

import re
from typing import Any
from uuid import uuid4

from fastapi import APIRouter, Depends, Request
from pydantic import BaseModel, Field
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from app.core.dependencies import get_db
from app.core.rbac import Permission, get_request_role, require_permission
from app.core.tenant_context import (
    extract_tenant_header,
    multi_tenant_enabled,
    resolve_tenant_context,
    tenant_dict,
)
from app.models.tenant import Tenant, TenantMember
from app.utils.exceptions import AgentFlowError, NotFoundError

router = APIRouter()

_SLUG_RE = re.compile(r"^[a-z0-9]([a-z0-9-]{1,62}[a-z0-9])?$")

class TenantCreate(BaseModel):
    name: str = Field(..., min_length=1, max_length=200)
    slug: str = Field(..., min_length=2, max_length=100)
    plan_id: str | None = None

class TenantMemberAdd(BaseModel):
    user_id: str = Field(..., min_length=1, max_length=100)
    role: str = Field(default="member", max_length=32)

class TenantResponse(BaseModel):
    id: str
    name: str
    slug: str
    status: str
    plan_id: str | None = None

```

```

def _actor(request: Request) -> str:
    return getattr(request.state, "actor", None) or "anonymous"

def _slugify(raw: str) -> str:
    s = raw.strip().lower().replace("_", "-")
    s = re.sub(r"[^a-z0-9-]+", "-", s)
    s = re.sub(r"-+", "-", s).strip("-")
    return s[:64] or "tenant"

@router.get("/context")
async def tenant_context(
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    """Current multi-tenant resolution (header + membership)."""
    role = get_request_role(request)
    await resolve_tenant_context(
        session,
        actor=_actor(request),
        header_value=extract_tenant_header(request),
        system_role=role.value,
    )
    return tenant_dict()

@router.get("")
@require_permission(Permission.TENANT_MANAGE, Permission.TASK_READ, require_all=False)
async def list_tenants(
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    """List tenants visible to the caller.

    system_admin: all tenants.
    others: memberships only.
    """
    actor = _actor(request)
    role = get_request_role(request)
    if role.value in {"system_admin", "admin"}:
        rows = (
            (await session.execute(select(Tenant).order_by(Tenant.created_at.desc())))
            .scalars()
            .all()
        )
    else:
        q = (
            select(Tenant)
            .join(TenantMember, TenantMember.tenant_id == Tenant.id)
            .where(TenantMember.user_id == actor)
            .order_by(Tenant.created_at.desc())
        )
        rows = (await session.execute(q)).scalars().all()
    return {
        "items": [
            {
                "id": t.id,
                "name": t.name,
                "slug": t.slug,
                "status": t.status,
                "plan_id": t.plan_id,
            }
            for t in rows
        ],
        "multi_tenant_enabled": multi_tenant_enabled(),
    }

```

```

@router.post("", status_code=201)
@require_permission(Permission.TENANT_CREATE)
async def create_tenant(
    body: TenantCreate,
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    """Create a tenant; creator becomes tenant_admin member."""
    slug = _slugify(body.slug or body.name)
    if not _SLUG_RE.match(slug):
        raise AgentFlowError(
            message="Invalid tenant slug",
            status_code=422,
            detail="slug must be 2-64 chars [a-z0-9-]",
        )
    exists = await session.execute(select(Tenant).where(Tenant.slug == slug))
    if exists.scalar_one_or_none():
        raise AgentFlowError(
            message="Tenant slug already exists",
            status_code=409,
            detail=slug,
        )

    actor = _actor(request)
    tenant = Tenant(
        id=str(uuid4()),
        name=body.name.strip(),
        slug=slug,
        status="active",
        plan_id=body.plan_id,
    )
    session.add(tenant)
    session.add(
        TenantMember(
            id=str(uuid4()),
            tenant_id=tenant.id,
            user_id=actor,
            role="tenant_admin",
        )
    )
    await session.flush()
    return {
        "id": tenant.id,
        "name": tenant.name,
        "slug": tenant.slug,
        "status": tenant.status,
        "plan_id": tenant.plan_id,
        "member_role": "tenant_admin",
    }

@router.get("/{tenant_id}")
@require_permission(Permission.TASK_READ)
async def get_tenant(
    tenant_id: str,
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    """Get tenant if caller is member or system_admin."""
    role = get_request_role(request)
    result = await session.execute(
        select(Tenant).where((Tenant.id == tenant_id) | (Tenant.slug == tenant_id))
    )
    tenant = result.scalar_one_or_none()
    if tenant is None:
        raise NotFoundError("Tenant", tenant_id)

    if role.value not in {"system_admin", "admin"}:
        mem = await session.execute(
            select(TenantMember).where(

```



```

        TenantMember.tenant_id == tenant.id,
        TenantMember.user_id == _actor(request),
    )
)
if mem.scalar_one_or_none() is None:
    raise NotFoundError("Tenant", tenant_id)

return {
    "id": tenant.id,
    "name": tenant.name,
    "slug": tenant.slug,
    "status": tenant.status,
    "plan_id": tenant.plan_id,
}

@router.post("/{tenant_id}/members", status_code=201)
@require_permission(Permission.TENANT_MANAGE)
async def add_member(
    tenant_id: str,
    body: TenantMemberAdd,
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    """Add or update a tenant member (tenant_admin / system_admin)."""
    result = await session.execute(
        select(Tenant).where((Tenant.id == tenant_id) | (Tenant.slug == tenant_id))
    )
    tenant = result.scalar_one_or_none()
    if tenant is None:
        raise NotFoundError("Tenant", tenant_id)

    # Membership check unless system_admin
    role = get_request_role(request)
    if role.value not in {"system_admin", "admin"}:
        mem = await session.execute(
            select(TenantMember).where(
                TenantMember.tenant_id == tenant.id,
                TenantMember.user_id == _actor(request),
            )
        )
        m = mem.scalar_one_or_none()
        if m is None or m.role not in {"tenant_admin", "manager"}:
            raise AgentFlowError(
                message="Only tenant_admin/manager can add members",
                status_code=403,
            )

    allowed = {
        "tenant_admin",
        "manager",
        "reviewer",
        "member",
        "viewer",
        "user",
        "guest",
    }
    member_role = body.role.strip().lower()
    if member_role not in allowed:
        raise AgentFlowError(
            message="Invalid member role",
            status_code=422,
            detail=sorted(allowed),
        )

    # Normalize legacy
    if member_role == "user":
        member_role = "member"
    if member_role == "guest":
        member_role = "viewer"

```

```

existing = await session.execute(
    select(TenantMember).where(
        TenantMember.tenant_id == tenant.id,
        TenantMember.user_id == body.user_id.strip(),
    )
)
row = existing.scalar_one_or_none()
if row:
    row.role = member_role
    await session.flush()
    return {
        "tenant_id": tenant.id,
        "user_id": row.user_id,
        "role": row.role,
        "updated": True,
    }

row = TenantMember(
    id=str(uuid4()),
    tenant_id=tenant.id,
    user_id=body.user_id.strip(),
    role=member_role,
)
session.add(row)
await session.flush()
return {
    "tenant_id": tenant.id,
    "user_id": row.user_id,
    "role": row.role,
    "updated": False,
}

@router.get("/{tenant_id}/members")
@require_permission(Permission.TENANT_MANAGE, Permission.TASK_READ, require_all=False)
async def list_members(
    tenant_id: str,
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    result = await session.execute(
        select(Tenant).where((Tenant.id == tenant_id) | (Tenant.slug == tenant_id))
    )
    tenant = result.scalar_one_or_none()
    if tenant is None:
        raise NotFoundError("Tenant", tenant_id)

    rows = (
        (
            await session.execute(
                select(TenantMember).where(TenantMember.tenant_id == tenant.id)
            )
        )
        .scalars()
        .all()
    )
    return {
        "tenant_id": tenant.id,
        "items": [{"user_id": m.user_id, "role": m.role, "id": m.id} for m in rows],
    }

# === File: app/api/v1/endpoints/tools.py ===
# AgentFlow-Eval V1.0.0
"""Built-in tool registry listing for UI / docs."""

from __future__ import annotations

from typing import Any

from fastapi import APIRouter, Request
from pydantic import BaseModel, Field

```

```

from app.core.agent_runner.tool_sandbox import BUILTIN_TOOLS, run_tool_sandboxed
from app.core.rbac import Permission, require_permission

```

```

router = APIRouter()

```

```

class ToolProbeRequest(BaseModel):
    name: str = Field(..., description="Tool name")
    args: dict[str, Any] = Field(default_factory=dict)

```

```

@router.get("/")
@require_permission(Permission.TASK_READ)
async def list_tools(request: Request) -> dict[str, Any]:
    """List sandboxed built-in and plugin tools available to agents."""
    items = []
    for name, meta in sorted(BUILTIN_TOOLS.items()):
        items.append(
            {
                "name": name,
                "description": meta["description"],
                "parameters": meta["parameters"],
                "required": meta.get("required") or [],
                "sandbox": True,
                "network": False,
                "source": "builtin",
            }
        )
    try:
        from app.core.plugins.registry import get_capability_registry

        for t in get_capability_registry().list_tools():
            items.append(
                {
                    "name": t["name"],
                    "description": t.get("description") or "",
                    "parameters": t.get("parameters") or {},
                    "required": t.get("required") or [],
                    "sandbox": True,
                    "network": bool(t.get("network")),
                    "source": "plugin",
                    "plugin_id": t.get("plugin_id"),
                }
            )
    except Exception:
        pass
    return {"items": items, "total": len(items)}

```

```

@router.post("/probe")
@require_permission(Permission.SYSTEM_CONFIG)
async def probe_tool(request: Request, body: ToolProbeRequest) -> dict[str, Any]:
    """Execute a built-in tool once (dev/debug). Requires system:config."""
    output = run_tool_sandboxed(body.name, body.args)
    return {"name": body.name, "args": body.args, "output": output}
# === File: app/api/v1/endpoints/traces.py ===
# AgentFlow-Eval V1.0.0
"""执行轨迹查询接口 —— 列表、详情、评分与人工审核（含 actor 隔离）。"""

```

```

from __future__ import annotations

```

```

from typing import Any

```

```

from fastapi import APIRouter, Depends, Query, Request
from pydantic import BaseModel, Field
from sqlalchemy import func, select
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy.orm import selectinload

```

```

from app.core.dependencies import get_db
from app.core.rbac import Permission, require_permission
from app.core.tenancy import (
    apply_trace_owner_filter,
    load_task_for_suite,
    load_trace_with_access,
)
from app.models.metric_score import MetricScore
from app.models.test_suite import TestSuite
from app.models.trace import Trace
from app.schemas.trace import (
    JudgeResultResponse,
    MetricScoreResponse,
    TraceListResponse,
    TraceResponse,
)
from app.utils.exceptions import NotFoundError

router = APIRouter()

class HumanReviewRequest(BaseModel):
    """人工审核评分请求体。"""

    metric_name: str = Field(..., description="要审核的指标名称")
    human_score: float = Field(..., ge=0, le=100, description="人工评分 (0-100)")
    reviewer: str = Field(..., min_length=1, max_length=100, description="审核人标识")
    reason: str = Field("", description="审核备注")

class HumanReviewResponse(BaseModel):
    """人工审核评分响应。"""

    trace_id: str
    metric_name: str
    original_score: float
    human_score: float
    reviewer: str
    is_human_reviewed: bool

def _actor(request: Request) -> str:
    return getattr(request.state, "actor", None) or "anonymous"

@router.get("", response_model=TraceListResponse)
@require_permission(Permission.EVALUATION_READ)
async def list_traces(
    request: Request,
    test_suite_id: str | None = Query(None, description="按测试用例筛选"),
    page: int = Query(1, ge=1, description="页码"),
    page_size: int = Query(20, ge=1, le=100, description="每页条数"),
    session: AsyncSession = Depends(get_db),
) -> Any:
    """获取执行轨迹列表（按任务归属隔离）。"""
    actor = _actor(request)

    if test_suite_id:
        # Ensure caller can access the suite's parent task
        await load_task_for_suite(session, test_suite_id, actor)

    query = select(Trace).options(selectinload(Trace.metric_scores))
    count_query = select(func.count(Trace.id))

    query = apply_trace_owner_filter(query, actor)
    count_query = apply_trace_owner_filter(count_query, actor)

    if test_suite_id:
        query = query.where(Trace.test_suite_id == test_suite_id)
        count_query = count_query.where(Trace.test_suite_id == test_suite_id)

```

```

query = (
    query.order_by(Trace.created_at.desc())
    .offset((page - 1) * page_size)
    .limit(page_size)
)

total_result = await session.execute(count_query)
total = total_result.scalar() or 0

result = await session.execute(query)
traces = result.scalars().unique().all()

items = [_trace_to_response(t) for t in traces]
return TraceListResponse(items=items, total=total)

@router.get("/{trace_id}", response_model=TraceResponse)
@require_permission(Permission.EVALUATION_READ)
async def get_trace(
    trace_id: str,
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> Any:
    """获取单条执行轨迹（含评分），校验任务归属。"""
    trace = await load_trace_with_access(session, trace_id, _actor(request))
    return _trace_to_response(trace)

@router.post("/{trace_id}/judge", response_model=JudgeResultResponse)
@require_permission(Permission.EVALUATION_SUBMIT)
async def judge_trace(
    trace_id: str,
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> Any:
    """对指定轨迹执行 LLM-as-Judge 评分（需拥有所属任务）。

    Existing scores are served from a versioned 1h cache when present.
    """
    from app.core.cache.invalidation import invalidate_eval
    from app.core.cache.services import eval_version_from_scores, get_cached_eval_result

    trace = await load_trace_with_access(session, trace_id, _actor(request))

    if trace.metric_scores:
        version = eval_version_from_scores(list(trace.metric_scores))

    async def _from_db() -> dict:
        scores = {ms.metric_name: ms.score for ms in trace.metric_scores}
        total = sum(scores.values())
        reason = trace.metric_scores[0].reason if trace.metric_scores else ""
        token_cost = (
            (trace.metric_scores[0].extra_metadata or {}).get("token_cost", 0)
            if trace.metric_scores
            else 0
        )
    return {
        "scores": scores,
        "total": total,
        "reason": reason,
        "token_cost": token_cost,
    }

    payload = await get_cached_eval_result(trace_id, version, _from_db)
    return JudgeResultResponse(**payload)

suite_result = await session.execute(
    select(TestSuite).where(TestSuite.id == trace.test_suite_id)
)

```

```

suite = suite_result.scalar_one_or_none()
if not suite:
    raise NotFoundError("测试用例", trace.test_suite_id)

from app.core.celery_app.tasks import build_llm_judge
from app.core.judge_engine.scorecard import extract_judge_config_from_agent_config
from app.models.task import Task

judge_cfg: dict = {}
try:
    task_r = await session.execute(select(Task).where(Task.id == suite.task_id))
    parent_task = task_r.scalar_one_or_none()
    if parent_task is not None:
        judge_cfg = extract_judge_config_from_agent_config(
            getattr(parent_task, "agent_config", None) or {}
        )
except Exception:
    judge_cfg = {}

judge = build_llm_judge(judge_cfg)
judge_result = await judge.evaluate(
    trace_steps=trace.steps,
    expected_output=suite.expected_output,
    expected_tools=suite.expected_tools,
)
scores = (
    judge_result.get("scores", {})
    if isinstance(judge_result, dict)
    else judge_result.scores
)
total = (
    judge_result.get("total", 0.0)
    if isinstance(judge_result, dict)
    else judge_result.total
)
reason = (
    judge_result.get("reason", "")
    if isinstance(judge_result, dict)
    else judge_result.reason
)
token_cost = (
    judge_result.get("token_cost", 0)
    if isinstance(judge_result, dict)
    else judge_result.token_cost
)

for metric_name, score in scores.items():
    ms = MetricScore(
        trace_id=trace_id,
        metric_name=metric_name,
        score=score,
        reason=reason,
        extra_metadata={"token_cost": token_cost},
    )
    session.add(ms)

await session.commit()
await invalidate_eval(trace_id)

payload = {
    "scores": scores,
    "total": total,
    "reason": reason,
    "token_cost": token_cost,
}

# Seed versioned cache for subsequent reads
from app.core.cache.client import get_cache
from app.core.cache.keys import CacheTTL, eval_result_key

ver = eval_version_from_scores(

```

```

[
    MetricScore(
        trace_id=trace_id,
        metric_name=n,
        score=float(s),
        reason=reason,
    )
    for n, s in scores.items()
]
)
await get_cache().set(
    eval_result_key(trace_id, ver),
    payload,
    ttl=int(CacheTTL.EVAL_RESULT),
)

return JudgeResultResponse(**payload)

@router.post("/{trace_id}/review", response_model=HumanReviewResponse)
@require_permission(Permission.EVALUATION_APPROVE)
async def review_trace(
    trace_id: str,
    body: HumanReviewRequest,
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> Any:
    """提交人工审核评分（需拥有所属任务）。"""
    trace = await load_trace_with_access(session, trace_id, _actor(request))

    existing_ms = None
    for ms in trace.metric_scores:
        if ms.metric_name == body.metric_name:
            existing_ms = ms
            break

    if existing_ms:
        original_score = existing_ms.score
        existing_ms.human_score = body.human_score
        existing_ms.is_human_reviewed = True
        existing_ms.reviewer = body.reviewer
        if body.reason:
            existing_ms.reason = body.reason
    else:
        original_score = 0.0
        new_ms = MetricScore(
            trace_id=trace_id,
            metric_name=body.metric_name,
            score=0.0,
            human_score=body.human_score,
            is_human_reviewed=True,
            reviewer=body.reviewer,
            reason=body.reason or "Human review",
        )
        session.add(new_ms)

    await session.commit()
    try:
        from app.core.cache.invalidation import invalidate_eval

        await invalidate_eval(trace_id)
    except Exception:
        pass

    return HumanReviewResponse(
        trace_id=trace_id,
        metric_name=body.metric_name,
        original_score=original_score,
        human_score=body.human_score,
        reviewer=body.reviewer,

```

```

        is_human_reviewed=True,
    )

def _trace_to_response(trace: Trace) -> TraceResponse:
    """将 Trace ORM 对象转换为响应模型（含版本 / Token / 成本可观测字段）。"""
    from app.schemas.trace import TokenUsage

    prompt_tokens = int(getattr(trace, "prompt_tokens", 0) or 0)
    completion_tokens = int(getattr(trace, "completion_tokens", 0) or 0)
    total_tokens = int(trace.total_tokens or 0)
    return TraceResponse(
        id=trace.id,
        test_suite_id=trace.test_suite_id,
        user_query=trace.user_query,
        steps=trace.steps or [],
        total_tokens=total_tokens,
        response_time_ms=trace.response_time_ms,
        status=trace.status.value,
        created_at=trace.created_at,
        metric_scores=[
            MetricScoreResponse(
                id=ms.id,
                metric_name=ms.metric_name,
                score=ms.score,
                reason=ms.reason,
                extra_metadata=ms.extra_metadata,
            )
            for ms in (trace.metric_scores or [])
        ],
        prompt_tokens=prompt_tokens,
        completion_tokens=completion_tokens,
        cost=float(getattr(trace, "cost", 0.0) or 0.0),
        agent_version=getattr(trace, "agent_version", None),
        prompt_version=getattr(trace, "prompt_version", None),
        model_version=getattr(trace, "model_version", None),
        tool_version=getattr(trace, "tool_version", None),
        token_usage=TokenUsage(
            prompt_tokens=prompt_tokens,
            completion_tokens=completion_tokens,
            total_tokens=total_tokens or (prompt_tokens + completion_tokens),
        ),
    )

# == File: app/api/v1/endpoints/ws.py ==
# AgentFlow-Eval V1.0.0
"""WebSocket endpoint for live task activity."""

from __future__ import annotations

import asyncio
import logging

from fastapi import APIRouter, WebSocket, WebSocketDisconnect

from app.core.ws_hub import manager

logger = logging.getLogger(__name__)

router = APIRouter()

@router.websocket("/ws/activities")
async def activities_ws(websocket: WebSocket) -> None:
    """Push task status events to the browser.

    Protocol:
    - Server -> client: JSON task_status events
    - Client -> server: optional `{"type": "ping"}` -> `{"type": "pong"}`
    - On connect: `{"type": "hello", "clients": N}`
    """

```



```

await manager.connect(websocket)
try:
    await websocket.send_json(
        {"type": "hello", "clients": manager.count, "mode": "ws"}
    )
    while True:
        try:
            data = await asyncio.wait_for(websocket.receive_text(), timeout=45.0)
        except asyncio.TimeoutError:
            # keepalive
            try:
                await websocket.send_json({"type": "ping"})
            except Exception:
                break
            continue
        if not data:
            continue
        if "ping" in data.lower():
            await websocket.send_json({"type": "pong"})
    except WebSocketDisconnect:
        pass
    except Exception as exc:
        logger.debug("WS closed: %s", exc)
    finally:
        await manager.disconnect(websocket)
# === File: app/utils/__init__.py ===
# AgentFlow-Eval V1.0.0
"""工具包。"""
# === File: app/utils/cost.py ===
# AgentFlow-Eval V1.0.0
"""成本计算工具 —— 基于模型和 Token 数量计算 API 调用费用。

```

```

用法:
from app.utils.cost import calculate_cost
cost = calculate_cost("gpt-4o-mini", prompt_tokens=1000, completion_tokens=500)
"""

```

```

from __future__ import annotations

```

```

from dataclasses import dataclass

```

```

@dataclass(frozen=True)
class ModelPricing:
    """每百万 Token 的价格 (USD)。"""

    prompt_per_million: float
    completion_per_million: float

```

```

# 官方定价表 (截至 2025-Q2, 请定期更新)
# https://openai.com/pricing
MODEL_PRICING: dict[str, ModelPricing] = {
    # GPT-4o 系列
    "gpt-4o": ModelPricing(2.50, 10.00),
    "gpt-4o-2024-11-20": ModelPricing(2.50, 10.00),
    "gpt-4o-2024-08-06": ModelPricing(2.50, 10.00),
    "gpt-4o-2024-05-13": ModelPricing(5.00, 15.00),
    # GPT-4o-mini 系列
    "gpt-4o-mini": ModelPricing(0.15, 0.60),
    "gpt-4o-mini-2024-07-18": ModelPricing(0.15, 0.60),
    # GPT-4.1 系列
    "gpt-4.1": ModelPricing(2.00, 8.00),
    "gpt-4.1-mini": ModelPricing(0.40, 1.60),
    "gpt-4.1-nano": ModelPricing(0.10, 0.40),
    # o 系列推理模型
    "o4-mini": ModelPricing(1.10, 4.40),
    "o3": ModelPricing(2.00, 8.00),
    "o3-mini": ModelPricing(1.10, 4.40),
    "o1": ModelPricing(15.00, 60.00),

```

```

        "ol-mini": ModelPricing(1.10, 4.40),
        # GPT-4 Turbo (legacy)
        "gpt-4-turbo": ModelPricing(10.00, 30.00),
        "gpt-4-turbo-preview": ModelPricing(10.00, 30.00),
        # GPT-3.5 (legacy)
        "gpt-3.5-turbo": ModelPricing(0.50, 1.50),
        "gpt-3.5-turbo-0125": ModelPricing(0.50, 1.50),
    }

# 默认 fallback 价格 (使用 gpt-4o-mini 作为保守估算)
_DEFAULT_PRICING = ModelPricing(0.15, 0.60)

def calculate_cost(
    model: str,
    prompt_tokens: int,
    completion_tokens: int,
) -> float:
    """计算单次 API 调用的费用 (USD)。


Args:


    model: 模型名称 (如 "gpt-4o-mini")。
    prompt_tokens: 输入 Token 数。
    completion_tokens: 输出 Token 数。


Returns:


    费用 (USD)，保留 8 位小数以支持微量计费。


Examples:


    >>> calculate_cost("gpt-4o-mini", 1000, 500)
    0.00045
    >>> calculate_cost("gpt-4o", 10000, 5000)
    0.075
    """
    pricing = MODEL_PRICING.get(model, _DEFAULT_PRICING)
    cost = (prompt_tokens / 1_000_000) * pricing.prompt_per_million + (
        completion_tokens / 1_000_000
    ) * pricing.completion_per_million
    return round(cost, 8)

def get_supported_models() -> list[str]:
    """返回所有已配置定价的模型列表。"""
    return sorted(MODEL_PRICING.keys())
# === File: app/utils/exceptions.py ===
# AgentFlow-Eval V1.0.0
"""Custom exception hierarchy and unified error response format."""

from __future__ import annotations
from typing import Any
from datetime import datetime, timezone

class AgentFlowError(Exception):
    """Base exception for all application errors."""

    def __init__(
        self,
        message: str = "Internal error",
        status_code: int = 500,
        detail: Any = None,
    ):
        self.message = message
        self.status_code = status_code
        self.detail = detail
        super().__init__(self.message)

class NotFoundError(AgentFlowError):
    """Resource not found (HTTP 404)."""

```

```

def __init__(self, resource: str = "Resource", resource_id: str = ""):
    msg = (
        f"{resource} not found: {resource_id}"
        if resource_id
        else f"{resource} not found"
    )
    super().__init__(message=msg, status_code=404)

class ValidationError(AgentFlowError):
    """Request validation failed (HTTP 422)."""

    def __init__(self, message: str = "Validation failed", detail: Any = None):
        super().__init__(message=message, status_code=422, detail=detail)

class BusinessError(AgentFlowError):
    """Business logic error (HTTP 400)."""

    def __init__(self, message: str = "Business error", detail: Any = None):
        super().__init__(message=message, status_code=400, detail=detail)

class TaskStateError(AgentFlowError):
    """Task state conflict (HTTP 409)."""

    def __init__(self, message: str = "Task state error"):
        super().__init__(message=message, status_code=409)

class ExternalServiceError(AgentFlowError):
    """External service unavailable (HTTP 503)."""

    def __init__(self, message: str = "External service error", detail: Any = None):
        super().__init__(message=message, status_code=503, detail=detail)

class RateLimitError(AgentFlowError):
    """Rate limit exceeded (HTTP 429)."""

    def __init__(self, message: str = "Too many requests", detail: Any = None):
        super().__init__(message=message, status_code=429, detail=detail)

class ForbiddenError(AgentFlowError):
    """Permission denied (HTTP 403)."""

    def __init__(self, message: str = "Forbidden", detail: Any = None):
        super().__init__(message=message, status_code=403, detail=detail)

def error_response(
    status_code: int,
    message: str,
    detail: Any = None,
    request_id: str | None = None,
    stacktrace: str | None = None,
    error_id: str | None = None,
) -> dict[str, Any]:
    """Generate unified JSON error response.

    Returns dict with code, message, detail, timestamp, request_id,
    optional error_id (support correlation), and stacktrace (dev only).
    """
    resp: dict[str, Any] = {
        "error": {
            "code": status_code,
            "message": message,
            "timestamp": datetime.now(timezone.utc).isoformat(),

```

```

    }
}
if detail:
    resp["error"]["detail"] = detail
if request_id:
    resp["error"]["request_id"] = request_id
if error_id:
    resp["error"]["error_id"] = error_id
if stacktrace:
    resp["error"]["stacktrace"] = stacktrace
return resp
# === File: app/utils/logger.py ===
# AgentFlow-Eval V1.0.0
"""Application logging entrypoint — delegates to AOLS (structlog JSON)."""

from __future__ import annotations

from typing import Any

def setup_logging() -> None:
    """Configure process-wide logging (structlog + rotating file)."""
    from app.core.observability.aols.logger import setup_aols_logging

    setup_aols_logging()

def get_logger(name: str | None = None) -> Any:
    """Structured logger factory (compat re-export)."""
    from app.core.observability.aols.logger import get_logger as _get

    return _get(name)
# === File: app/plugins/__init__.py ===
# AgentFlow-Eval V1.0.0
"""Built-in / example plugins package (optional load via PLUGIN_MODULES)."""
# === File: app/plugins/examples/__init__.py ===
# AgentFlow-Eval V1.0.0
"""Example plugins demonstrating AgentRunner / Judge / Tool / Hook types."""
# === File: app/plugins/examples/audit_hooks.py ===
# AgentFlow-Eval V1.0.0
"""Example HOOK plugin — records pre/post pipeline events in-memory."""

from __future__ import annotations

from typing import Any

from app.core.plugins.base import (
    HOOK_POST_AGENT_RUN,
    HOOK_POST_JUDGE,
    HOOK_PRE_AGENT_RUN,
    HOOK_PRE_JUDGE,
    BasePlugin,
    PluginContext,
    PluginMeta,
    PluginType,
)

# Module-level ring buffer for tests / debugging
_EVENT_LOG: list[dict[str, Any]] = []
_MAX = 200

def get_event_log() -> list[dict[str, Any]]:
    return list(_EVENT_LOG)

def clear_event_log() -> None:
    _EVENT_LOG.clear()

```

```

def _append(event: str, payload: dict[str, Any]) -> dict[str, Any]:
    item = {"event": event, **{k: v for k, v in payload.items() if k != "event"}}
    _EVENT_LOG.append(item)
    if len(_EVENT_LOG) > _MAX:
        del _EVENT_LOG[: len(_EVENT_LOG) - _MAX]
    return item

class Plugin(BasePlugin):
    meta = PluginMeta(
        name="audit_hooks",
        version="1.0.0",
        plugin_type=PluginType.HOOK,
        description="Logs pre/post agent and judge hooks (example).",
        author="AgentFlow-Eval",
        provides=[],
    )

    def on_activate(self, ctx: PluginContext) -> None:
        assert ctx.register_hook is not None

    def pre_agent(payload: dict[str, Any]) -> dict[str, Any]:
        return _append(HOOK_PRE_AGENT_RUN, payload)

    def post_agent(payload: dict[str, Any]) -> dict[str, Any]:
        return _append(HOOK_POST_AGENT_RUN, payload)

    def pre_judge(payload: dict[str, Any]) -> dict[str, Any]:
        return _append(HOOK_PRE_JUDGE, payload)

    def post_judge(payload: dict[str, Any]) -> dict[str, Any]:
        return _append(HOOK_POST_JUDGE, payload)

    ctx.register_hook(HOOK_PRE_AGENT_RUN, pre_agent, priority=50)
    ctx.register_hook(HOOK_POST_AGENT_RUN, post_agent, priority=50)
    ctx.register_hook(HOOK_PRE_JUDGE, pre_judge, priority=50)
    ctx.register_hook(HOOK_POST_JUDGE, post_judge, priority=50)
# === File: app/plugins/examples/echo_runner.py ===
# AgentFlow-Eval V1.0.0
"""Example AGENT_RUNNER plugin — deterministic echo runner."""

from __future__ import annotations

from datetime import datetime, timezone
from typing import Any

from app.core.agent_runner.base import AgentResult, BaseAgentRunner
from app.core.plugins.base import BasePlugin, PluginContext, PluginMeta, PluginType

class EchoAgentRunner(BaseAgentRunner):
    """Returns the user query as a single final answer step (no LLM)."""

    async def run(
        self,
        query: str,
        tools: list[Any] | None = None,
        *,
        agent_config: dict[str, Any] | None = None,
    ) -> AgentResult:
        _ = tools
        cfg = dict(agent_config or {})
        prefix = str(cfg.get("prefix") or "ECHO: ")
        text = f"{prefix}{query}"
        steps = [
            {
                "type": "thought",
                "content": "Echo runner reflects the query without external calls.",
            },
        ]

```

```

        "type": "final",
        "content": text,
        "answer": text,
    },
]
return AgentResult(
    steps=steps,
    total_tokens=0,
    response_time_ms=1,
    status="success",
    final_answer=text,
    runner="echo",
    finished_at=datetime.now(timezone.utc),
)

class Plugin(BasePlugin):
    meta = PluginMeta(
        name="echo_runner",
        version="1.0.0",
        plugin_type=PluginType.AGENT_RUNNER,
        description="Deterministic echo AgentRunner (example).",
        author="AgentFlow-Eval",
        provides=["echo", "echo_runner"],
    )

    def on_activate(self, ctx: PluginContext) -> None:
        assert ctx.register_runner is not None

        def factory(agent_config: dict[str, Any]) -> EchoAgentRunner:
            return EchoAgentRunner()

        ctx.register_runner("echo", factory)
        ctx.register_runner("echo_runner", factory)
# === File: app/plugins/examples/echo_tool.py ===
# AgentFlow-Eval V1.0.0
"""Example TOOL plugin — echo arguments as JSON."""

from __future__ import annotations

import json
from typing import Any

from app.core.plugins.base import BasePlugin, PluginContext, PluginMeta, PluginType

def _echo_tool(**kwargs: Any) -> str:
    return json.dumps({"echo": kwargs}, ensure_ascii=False)

class Plugin(BasePlugin):
    meta = PluginMeta(
        name="echo_tool",
        version="1.0.0",
        plugin_type=PluginType.TOOL,
        description="Echo tool arguments as JSON (example).",
        author="AgentFlow-Eval",
        provides=["echo"],
    )

    def on_activate(self, ctx: PluginContext) -> None:
        assert ctx.register_tool is not None
        ctx.register_tool(
            "echo",
            _echo_tool,
            description="Echo back all tool arguments as a JSON object.",
            parameters={
                "message": {
                    "type": "string",
                    "description": "Optional message to echo",

```

```

        },
    },
    required=[],
    network=False,
)
# === File: app/plugins/examples/length_judge.py ===
# AgentFlow-Eval V1.0.0
"""Example JUDGE plugin — score based on answer length similarity."""

from __future__ import annotations

from typing import Any

from app.core.judge_engine.base import BaseJudge, JudgeResult
from app.core.plugins.base import BasePlugin, PluginContext, PluginMeta, PluginType

class LengthJudge(BaseJudge):
    """Simple deterministic judge for demos and tests."""

    async def evaluate(
        self,
        trace_steps: list[dict[str, Any]],
        expected_output: str,
        expected_tools: list[str],
    ) -> JudgeResult:
        # Extract last assistant / final answer-ish text
        answer = ""
        for step in reversed(trace_steps or []):
            if not isinstance(step, dict):
                continue
            for key in ("content", "output", "answer", "observation"):
                if step.get(key):
                    answer = str(step[key])
                    break
        if answer:
            break

        exp = (expected_output or "").strip()
        ans = answer.strip()
        if not exp and not ans:
            score = 1.0
        elif not exp or not ans:
            score = 0.0
        else:
            ratio = min(len(ans), len(exp)) / max(len(ans), len(exp))
            score = round(float(ratio), 4)

        tools_used = []
        for step in trace_steps or []:
            if isinstance(step, dict) and step.get("tool"):
                tools_used.append(str(step["tool"]))
            if isinstance(step, dict) and step.get("tool_name"):
                tools_used.append(str(step["tool_name"]))

        tool_score = 1.0
        if expected_tools:
            hit = sum(1 for t in expected_tools if t in tools_used)
            tool_score = hit / len(expected_tools)

        total = round(0.7 * score + 0.3 * tool_score, 4)
        return JudgeResult(
            scores={
                "length_similarity": score,
                "tool_coverage": tool_score,
            },
            total=total,
            reason=f"length_ratio={score}, tool_coverage={tool_score}",
            token_cost=0,
        )

```

```

class Plugin(BasePlugin):
    meta = PluginMeta(
        name="length_judge",
        version="1.0.0",
        plugin_type=PluginType.JUDGE,
        description="Rule judge: answer length ratio + tool coverage.",
        author="AgentFlow-Eval",
        provides=["length", "length_judge"],
    )

    def on_activate(self, ctx: PluginContext) -> None:
        assert ctx.register_judge is not None

        def factory() -> LengthJudge:
            return LengthJudge()

        ctx.register_judge("length", factory)
        ctx.register_judge("length_judge", factory)
# === File: scripts/backfill_created_by.py ===
"""One-off SQLite backfill for tasks.created_by."""
from sqlalchemy import create_engine, text

engine = create_engine("sqlite:///./agentflow_eval.db")
with engine.begin() as conn:
    cols = {r[1] for r in conn.execute(text("PRAGMA table_info(tasks)").fetchall())}
    if "created_by" not in cols:
        conn.execute(
            text(
                "ALTER TABLE tasks ADD COLUMN created_by "
                "VARCHAR(100) NOT NULL DEFAULT 'anonymous'"
            )
        )
        print("added created_by")
    else:
        print("created_by already present")
# === File: scripts/export_openapi.py ===
# AgentFlow-Eval V1.0.0
"""Export FastAPI OpenAPI schema and optionally check against frozen contract.

Usage:
cd backend
python scripts/export_openapi.py -o ../docs/openapi-v1.json
python scripts/export_openapi.py --check ../docs/openapi-v1.json
"""

from __future__ import annotations

import argparse
import json
import sys
from pathlib import Path

def _app():
    # Ensure backend root on path
    root = Path(__file__).resolve().parents[1]
    if str(root) not in sys.path:
        sys.path.insert(0, str(root))
    from app.main import app

    return app

def export_schema() -> dict:
    return _app().openapi()

def path_methods(schema: dict) -> set[tuple[str, str]]:

```



```

out: set[tuple[str, str]] = set()
for path, item in (schema.get("paths") or {}).items():
    if not isinstance(item, dict):
        continue
    for method in item:
        if method.lower() in {
            "get",
            "post",
            "put",
            "patch",
            "delete",
            "head",
            "options",
        }:
            out.add((path, method.lower()))
return out

def check_compatible(frozen: dict, current: dict) -> list[str]:
    """Return list of breaking-change messages (empty = ok).

    Breaking: frozen (path, method) missing from current.
    Additive endpoints are allowed.
    """
    errors: list[str] = []
    frozen_pm = path_methods(frozen)
    current_pm = path_methods(current)
    missing = frozen_pm - current_pm
    for path, method in sorted(missing):
        errors.append(f"BREAKING: removed {method.upper()} {path}")
    return errors

def main(argv: list[str] | None = None) -> int:
    parser = argparse.ArgumentParser(description="OpenAPI export / freeze check")
    parser.add_argument(
        "-o",
        "--output",
        type=Path,
        help="Write OpenAPI JSON to this path",
    )
    parser.add_argument(
        "--check",
        type=Path,
        metavar="FROZEN_JSON",
        help="Fail if current schema removes paths from frozen JSON",
    )
    args = parser.parse_args(argv)

    schema = export_schema()
    if args.output:
        args.output.parent.mkdir(parents=True, exist_ok=True)
        args.output.write_text(
            json.dumps(schema, ensure_ascii=False, indent=2) + "\n",
            encoding="utf-8",
        )
        print(f"Wrote {args.output} ({len(path_methods(schema))} operations)")

    if args.check:
        frozen = json.loads(args.check.read_text(encoding="utf-8"))
        errors = check_compatible(frozen, schema)
        if errors:
            print("API compatibility check FAILED:", file=sys.stderr)
            for e in errors:
                print(f"  - {e}", file=sys.stderr)
            return 1
        print(
            f"API compatibility check OK "
            f"(frozen={len(path_methods(frozen))} ops, "
            f"current={len(path_methods(schema))} ops)"

```

```

    )

    if not args.output and not args.check:
        # dump to stdout
        json.dump(schema, sys.stdout, ensure_ascii=False, indent=2)
        print()
    return 0

if __name__ == "__main__":
    raise SystemExit(main())
# === File: _start_api.py ===
import os, subprocess, sys
from pathlib import Path
env = os.environ.copy()
# parse .env.docker
p = Path("backend/.env.docker") if Path("backend/.env.docker").exists() else Path(".env.docker")
for line in p.read_text(encoding="utf-8").splitlines():
    line=line.strip()
    if not line or line.startswith("#") or "=" not in line: continue
    k,v=line.split("=",1)
    env[k.strip()]=v.strip()
# force host connectivity to docker published ports
pu=env.get("POSTGRES_USER","agentflow")
pp=env.get("POSTGRES_PASSWORD","")
pd=env.get("POSTGRES_DB","agentflow_eval")
env["DATABASE_URL"]=f"postgresql+asyncpg://{pu}:{pp}@127.0.0.1:5432/{pd}"
env["REDIS_URL"]="redis://127.0.0.1:6379/0"
env["CELERY_BROKER_URL"]=env["REDIS_URL"]
env["CELERY_RESULT_BACKEND"]=env["REDIS_URL"]
env["CELERY_TASK_ALWAYS_EAGER"]="true"
env["TASK_QUEUE_BACKEND"]="eager"
env["DEPLOY_PROFILE"]="lite"
env["ENV"]="dev"
env["DEBUG"]="true"
env["AUTH_ENABLED"]="false"
env["BILLING_ENABLED"]="false"
env["LOG_DB_SINK"]="true"
env["PYTHONPATH"]=str(Path(".").resolve())
os.chdir(str(Path(".").resolve()))
log=open("uvicorn-docker-db.log","w",encoding="utf-8")
proc=subprocess.Popen([sys.executable,"-m","uvicorn","app.main:app","--host","127.0.0.1","--port","8000"], env=env, st..)
print(proc.pid)
# === File: scripts_local_api_test.py ===
"""API functional smoke against local backend + Docker Postgres."""
from __future__ import annotations

import json
import time
import urllib.error
import urllib.request
from pathlib import Path

BASE = "http://127.0.0.1:8000"
API = BASE + "/api/v1"
results: list[dict] = []

def req(name: str, method: str, path: str, body=None) -> tuple[int, str]:
    if path.startswith("http"):
        url = path
    elif path.startswith("/health") or path.startswith("/metrics"):
        url = BASE + path
    else:
        url = API + path
    data = None
    headers: dict[str, str] = {}
    if body is not None:
        data = json.dumps(body).encode("utf-8")
        headers["Content-Type"] = "application/json"

```

```

request = urllib.request.Request(url, data=data, headers=headers, method=method)
try:
    with urllib.request.urlopen(request, timeout=90) as resp:
        code = resp.status
        text = resp.read().decode("utf-8", errors="replace")
        ok = 200 <= code < 300
        results.append({"name": name, "code": code, "ok": ok, "detail": text[:200]})
        return code, text
except urllib.error.HTTPError as e:
    text = e.read().decode("utf-8", errors="replace")
    results.append({"name": name, "code": e.code, "ok": False, "detail": text[:200]})
    return e.code, text
except Exception as e: # noqa: BLE001
    results.append({"name": name, "code": 0, "ok": False, "detail": str(e)[:200]})
    return 0, str(e)

def main() -> None:
    for _ in range(30):
        code, _ = req("probe", "GET", "/health/ready")
        if code == 200:
            results.clear()
            break
        time.sleep(0.5)

checks = [
    ("health", "GET", "/health"),
    ("health_live", "GET", "/health/live"),
    ("health_ready", "GET", "/health/ready"),
    ("metrics", "GET", "/metrics"),
    ("me", "GET", "/me"),
    ("dashboard", "GET", "/dashboard?days=7"),
    ("dashboard_stats", "GET", "/dashboard/stats"),
    ("tasks_list", "GET", "/tasks?page=1&page_size=10"),
    ("tools", "GET", "/tools"),
    ("settings", "GET", "/settings"),
    ("audit", "GET", "/audit?page=1&page_size=5"),
    ("traces", "GET", "/traces?page=1&page_size=5"),
    ("logs", "GET", "/logs?page=1&page_size=10"),
    ("logs_stats", "GET", "/logs/statistics?days=7"),
    ("obs_kpis", "GET", "/observability/kpis?days=7"),
    ("obs_slow", "GET", "/observability/slow-tasks?limit=10"),
    ("obs_topo", "GET", "/observability/error-topology?days=7"),
    ("diagnosis_list", "GET", "/diagnosis?limit=10"),
    ("plugins", "GET", "/plugins"),
    ("billing_plans", "GET", "/billing/plans"),
    ("billing_quota", "GET", "/billing/quota"),
    ("ab_list", "GET", "/ab"), # list is GET /ab (not /ab/experiments)
    ("experiments", "GET", "/experiments"),
    ("media_list", "GET", "/media"),
]

for name, method, path in checks:
    req(name, method, path)

code, text = req(
    "task_create",
    "POST",
    "/tasks",
    {
        "name": "容器栈联调任务",
        "description": "Docker Postgres + Celery",
        "agent_config": {
            "model": "gpt-4o-mini",
            "temperature": 0,
            "max_iterations": 3,
        },
    },
)
tid = None
try:

```

```

        tid = json.loads(text).get("id")
    except Exception:
        tid = None

    if tid:
        # body must be a JSON array of suite dicts
        req(
            "task_suites",
            "POST",
            f"/tasks/{tid}/test-suites",
            [
                {
                    "user_query": "1+1等于几? ",
                    "expected_output": "2",
                    "expected_tools": [],
                }
            ],
        )
        req("task_get", "GET", f"/tasks/{tid}")
        # Celery async: should return quickly when not eager
        req("task_execute", "POST", f"/tasks/{tid}/execute", {})
        # poll status up to ~60s
        for i in range(12):
            time.sleep(5)
            c, t = req(f"task_poll_{i}", "GET", f"/tasks/{tid}")
            try:
                st = json.loads(t).get("status")
            except Exception:
                st = None
            if st in {"completed", "failed", "cancelled", "timeout"}:
                break
        req("task_report", "GET", f"/reports/{tid}")
        req("diagnosis_task", "GET", f"/diagnosis/{tid}")

_, text = req("tasks_for_demo", "GET", "/tasks?page=1&page_size=20")
try:
    items = json.loads(text).get("items") or []
    demo = next((t for t in items if "Demo" in (t.get("name") or "")), None)
    if demo:
        req("diagnosis_demo", "GET", f"/diagnosis/{demo['id']}")
except Exception:
    pass

_, text = req("traces_sample", "GET", "/traces?page=1&page_size=1")
try:
    items = json.loads(text).get("items") or []
    if items:
        req("trace_detail", "GET", f"/traces/{items[0]['id']}")
except Exception:
    pass

final = [r for r in results if r["name"] not in {"probe", "tasks_for_demo", "traces_sample"}]
pass_n = sum(1 for r in final if r["ok"])
fail_n = sum(1 for r in final if not r["ok"])
print(f"PASS={pass_n} FAIL={fail_n} TOTAL={len(final)}")
for r in final:
    flag = "OK" if r["ok"] else "FAIL"
    print(f"{flag:4} {r['code']:3} {r['name']:18} {r['detail'][:90]}")

out = Path(__file__).resolve().parents[1] / "docs" / "_docker_api_test_results.json"
out.parent.mkdir(parents=True, exist_ok=True)
out.write_text(
    json.dumps({"pass": pass_n, "fail": fail_n, "results": final}, ensure_ascii=False, indent=2),
    encoding="utf-8",
)
print("wrote", out)

if __name__ == "__main__":
    main()

```